UNIVERSIDADE FEDERAL DO RIO DE JANEIRO
INSTITUTO DE COMPUTAÇÃO
INSTITUTO TÉRCIO PACITTI DE APLICAÇÕES E PESQUISAS COMPUTACIONAIS
PROGRAMA DE PÓS-GRADUAÇÃO EM INFORMÁTICA (PPGI)

BRUNO LLACER TROTTI

A COMPOSITIONAL AND DIAGRAMMATIC SYNTAX FOR PARAMETRIC LINEAR PROGRAMMING

RIO DE JANEIRO
2026

Bruno Llacer Trotti

A COMPOSITIONAL AND DIAGRAMMATIC SYNTAX FOR PARAMETRIC LINEAR PROGRAMMING

Dissertação de Mestrado apresentada ao Programa de Pós-Graduação em Informática, Instituto de Computação e Instituto Tércio Pacitti de Aplicações e Pesquisas Computacionais, Universidade Federal do Rio de Janeiro, como requisito parcial à obtenção do título de Mestre em Informática.

Orientador: Prof. João Paixão, D.Sc.

Rio de Janeiro
2026

Bruno Llacer Trotti

A COMPOSITIONAL AND DIAGRAMMATIC SYNTAX FOR PARAMETRIC LINEAR PROGRAMMING

Dissertação de Mestrado apresentada ao Programa de Pós-Graduação em Informática, Instituto de Computação e Instituto Tércio Pacitti de Aplicações e Pesquisas Computacionais, Universidade Federal do Rio de Janeiro, como requisito parcial à obtenção do título de Mestre em Informática.

Aprovado em: de de 2026

Prof. João Paixão, D.Sc., PPGI/UFRJ (Presidente)

Prof(a). TODO — Membro da Banca, D.Sc., Instituição

Prof(a). TODO — Membro da Banca, D.Sc., Instituição

STATEMENT ON THE USE OF ARTIFICIAL INTELLIGENCE

The use of AI in this paper is described in a few different ways. When it comes to the writing of the thesis it was restricted to the following use:

- Tikz Writing: Convert tables and diagrams from handwritten images to tikz syntax.
- Writing review: AI searched for grammatical inconsistencies, typos, open-ended and poorly-formed sentences.
- Review the mathematical argument: The AI was instructed to look for loops or failed arguments across mathematical proofs and derivations.
- Loop Review: Read and evaluate the work multiple times and provide critics and feedback about the overall writing.
- Latex Formatting: AI was used to adapt the paper to the necessary standards demmanded by a dissertation. This refers only to document structure(font size, title format, citations etc) not the writing content.

AI most relevant effort came in into the appendix for helping writing down a long list of well stablished mathematical definitions to provide the reader the necessary background. Text outside definitions are not a product of AI use, AI only wrote the definitions itself. The main body of the work (from introduction to conclusion) is almost entirely written by hand having AI only as a default reviewer.

During the development and research of the thesis, AI was used as a learning gate and searching mechanism to discuss ideas, look for references, investigate topics and understand papers and works from others.

1 LINEAR PROGRAMMING BACKGROUND

This chapter introduces the linear programming concepts needed to develop a compositional language for optimization. Readers already familiar with linear programming may proceed directly to Section 1.2, which specifies the definitions and conventions used in this work.

1.1 THE STRUCTURE OF LINEAR OPTIMIZATION

Optimization concerns the search for an object that minimizes or maximizes a specified criterion over a given set. Linear programs form an important class of optimization problems, with applications in engineering, economics, and other areas of applied mathematics.

An optimization problem has two components: constraints and an objective. The constraints define the *feasible region*, the set of admissible choices. For example, when travelling between cities, admissible routes may be restricted by a fuel budget, fuel consumption per kilometre, or the money available for refuelling. A route that violates these restrictions is infeasible.

To obtain a continuous linear program, we model a divisible amount of traffic rather than the choice of one indivisible car route. Consider three cities A, B, C, with roads $A \to B$, $B \to C$, and $A \to C$. Let $x_{AB}, x_{BC}, x_{AC} \geq 0$ be the amounts transported along these roads, let $p \geq 0$ be the amount to transport from A to C, and let $F \geq 0$ be the total fuel budget. Suppose each road consumes one unit of fuel per unit transported. The constraints are

$$\begin{aligned} x_{AB} + x_{AC} &= p, \\ x_{BC} &= x_{AB}, \\ x_{AB} + x_{BC} + x_{AC} &\leq F, \\ x_{AB}, x_{BC}, x_{AC} &\geq 0. \end{aligned}$$

The first two equations express conservation of traffic; the inequality expresses the fuel restriction. Requiring a single unsplit car route instead would require a discrete model, which is not the continuous LP considered here.

Several feasible transportation plans may reach the same destination. The objective provides a criterion for comparing them, such as monetary cost or travel time. An optimal solution is a feasible choice that achieves the best objective value, when such a choice exists.

For the continuous transportation model, suppose the monetary costs per unit on the three roads are 1, 2, and 4, respectively. The objective is

$$J(x_{AB}, x_{BC}, x_{AC}) = x_{AB} + 2x_{BC} + 4x_{AC}.$$

The route through B costs less per unit than the direct road, but consumes more fuel. Thus the objective and the fuel constraint describe different aspects of the same choice.

Together, the constraints and objective define the optimization problem: minimize the trans-

portation cost over the feasible flows.

Combining the objective and constraints gives the value function

$$V(p,F) = \inf_{x_{AB},x_{BC},x_{AC}\geq 0} \left\{ x_{AB} + 2x_{BC} + 4x_{AC} \;\middle|\; \begin{aligned} x_{AB} + x_{AC} &= p, \\ x_{BC} &= x_{AB}, \\ x_{AB} + x_{BC} + x_{AC} &\leq F \end{aligned} \right\}.$$

Here p and F are parameters, while the three road flows are decision variables. For example, $V(1,1) = 4$, since only the direct road is feasible, whereas $V(1,2) = 3$, since the cheaper route through B becomes feasible. If $p \geq 0$ and $F < p$, the problem is infeasible and its value is $+\infty$.

A linear program has linear equality and inequality constraints and a linear objective, possibly with a constant offset. The feasible region is a polyhedron: an intersection of finitely many closed half-spaces. A nonempty feasible region defined only by linear equalities is an affine subspace, but inequalities generally make it a polyhedron rather than an affine subspace.

Now consider a transportation problem planned in two stages. The first stage distributes the traffic leaving A, and the second stage forwards the traffic arriving at B to C. Let q be the amount sent through B and f the fuel allocated to the first stage. Their separate value functions are

$$U(p,q,f) = \inf_{x_{AB},x_{AC}\geq 0} \left\{ x_{AB} + 4x_{AC} \;\middle|\; \begin{aligned} x_{AB} + x_{AC} &= p, \\ x_{AB} &= q, \\ x_{AB} + x_{AC} &\leq f \end{aligned} \right\},$$

$$W(q,g) = \inf_{x_{BC}\geq 0} \{2x_{BC} \mid x_{BC} = q,\ x_{BC} \leq g\}.$$

Taking $g = F - f$, the original problem is recovered by connecting the stages and choosing their shared variables optimally:

$$V(p,F) = \inf_{q,f\in\mathbb{R}} \big(U(p,q,f) + W(q,F-f)\big).$$

The costs add, and the internal choices q and f are eliminated by taking an infimum. Indeed, the two fuel inequalities admit such an f exactly when $x_{AB} + x_{AC} + x_{BC} \leq F$. All other constraints and costs are those of the original problem. In this example the stage values are finite or $+\infty$, so their sum does not require a convention for mixed infinities.

Although this composition is straightforward, its notation requires explicit bookkeeping of interfaces, shared variables, and infima. Expanding U and W introduces the additional infima that define each stage.

We seek a language in which composition connects matching interfaces and the semantics performs the minimization over shared variables. Only the external interface remains exposed. This does not automatically compute the optimal value; rather, it removes the need to write the minimization explicitly each time two problems are composed. Such a language is *compositional*

by construction.

Furthermore, as we will see in the next chapters, "eliminating" the inner variables has a geometric interpretation. We keep the external values for which some feasible choice of the inner variables exists. This operation is called projection. In our example, this means keeping the pairs (p, F) that admit feasible choices of the flows and the shared variables q, f. For optimization, we also need to keep track of costs; we will explain how after introducing epigraphs. To reach that point, we need some formal definitions. The next section introduces the necessary notions for us.

1.2 NECESSARY NOTIONS

We use two descriptions of the same class of value functions: extended piecewise affine LP functions and right-hand side (RHS) parametric LP value functions. We begin with the finite-valued piecewise affine case.

For the geometric arguments below, the *epigraph* of a function $f : \mathbb{R}^k \to \overline{\mathbb{R}}$ is

$$\operatorname{epi}(f) := \{(p, t) \in \mathbb{R}^k \times \mathbb{R} \mid t \geq f(p)\}, \qquad \overline{\mathbb{R}} = \mathbb{R} \cup \{-\infty, +\infty\}.$$

At a point where $f(p) = +\infty$, there is no real t in the epigraph; where $f(p) = -\infty$, every real t is in it.

Definition 1.1 (Piecewise Affine Convex Function)**.** *A function $f : \mathbb{R}^n \to \mathbb{R}$ is called* piecewise affine convex *if there exist finitely many affine functions $a_i^\top x + b_i$, $i = 1, \ldots, k$, with $k \geq 1$, such that*

$$f(x) = \max_{i=1,\ldots,k} \{a_i^\top x + b_i\}.$$

Definition 1.2 (Extended Piecewise Affine LP Function)**.** *An* extended piecewise affine LP function *is a function $f : \mathbb{R}^k \to \overline{\mathbb{R}}$, where $\overline{\mathbb{R}} := \mathbb{R} \cup \{-\infty, +\infty\}$, of one of the following forms:*

1. *For a nonempty polyhedron $D \subseteq \mathbb{R}^k$,*

$$f(p) = \begin{cases} \max_{i=1,\ldots,r}(a_i^\top p + \beta_i), & p \in D, \\ +\infty, & p \notin D, \end{cases} \qquad r \geq 1.$$

2. *For a nonempty polyhedron $D \subseteq \mathbb{R}^k$,*

$$f(p) = \begin{cases} -\infty, & p \in D, \\ +\infty, & p \notin D. \end{cases}$$

3. *$f(p) = +\infty$ for every p.*

The first form extends the preceding definition to a restricted domain; the second and third forms account for unboundedness and infeasibility.

Definition 1.3 (RHS Parametric Linear Program)**.** *A* right-hand side parametric linear program *has value function*

$$v(p) := \inf_{x \in \mathbb{R}^n} \{c^\top x + d \mid Ax \geq Bp + e\},$$

where $A \in \mathbb{R}^{m \times n}$, $B \in \mathbb{R}^{m \times k}$, $c \in \mathbb{R}^n$, $e \in \mathbb{R}^m$, *and* $d \in \mathbb{R}$ *are fixed. Thus the parameter appears only on the right-hand side. We use* $\inf \varnothing = +\infty$ *for infeasibility and* $v(p) = -\infty$ *when the objective is unbounded below. The form* $Ax \geq p$ *is a special case.*

The following proposition shows that these definitions describe the same value functions. This equivalence specifies the intended semantics of the graphical language: extended piecewise affine LP functions represented by RHS parametric linear programs.

Proposition 1.1. *A function* $v : \mathbb{R}^k \to \overline{\mathbb{R}}$ *is an extended piecewise affine LP function if and only if it is the value function of a right-hand side parametric linear program as defined above.*

Proof. ($\Rightarrow$) Write the nonempty domain as $D = \{p \mid Hp \geq h\}$. For the first form, use the program

$$v(p) = \inf_{t \in \mathbb{R}} \{t \mid 0t \geq h - Hp, \quad t \geq a_i^\top p + \beta_i \text{ for } i = 1, \ldots, r\}.$$

For $p \in D$, the minimum is the largest affine expression; for $p \notin D$, the program is infeasible. For the second form, minimize a free variable z subject only to $0z \geq h - Hp$. This has value $-\infty$ on D and $+\infty$ outside it. The third form is represented by an inconsistent constraint $0z \geq 1$. All these programs have the required affine RHS form.

($\Leftarrow$) Let $v(p) = \inf_x \{c^\top x + d \mid Ax \geq Bp + e\}$. The feasible parameter domain

$$D = \{p \mid \exists x : \ Ax \geq Bp + e\}$$

is a polyhedron, since it is a projection of a polyhedron. Here projection means keeping the parameter coordinates and eliminating the decision variables. We use the standard facts that projections preserve polyhedra and that a feasible LP with an objective bounded below attains a finite optimum [Sch86]. If D is empty, v has the third form. Otherwise, the recession cone of every feasible set is $\{r \mid Ar \geq 0\}$, independently of p. Its elements are directions r such that moving from any feasible x to $x + \lambda r$ remains feasible for every $\lambda \geq 0$. By the standard recession criterion for linear programming [Sch86], the objective is unbounded below at a feasible parameter if and only if there exists r with $Ar \geq 0$ and $c^\top r < 0$. Such a direction makes it unbounded below at every parameter in D, giving the second form.

If no such direction exists, the optimum is finite and attained at every $p \in D$. Consequently,

$$\operatorname{epi}(v) = \{(p, t) \mid \exists x : \ Ax \geq Bp + e, \quad c^\top x + d \leq t\}$$

is a nonempty polyhedron. Write its defining inequalities as $u_j^\top p + \alpha_j t \geq \gamma_j$. Upward closure in t implies $\alpha_j \geq 0$. The inequalities with $\alpha_j = 0$ define the domain D; those with $\alpha_j > 0$ give

finitely many affine lower bounds on t. There is at least one such lower bound, since v is finite on D. Their maximum equals v on D, giving the first form. □

These descriptions support different forms of reasoning. An RHS parametric linear program specifies constraints and an objective, whereas an extended piecewise affine LP function describes the geometry of its value function. Different LP formulations may have the same value function. The intended graphical language represents both descriptions and allows diagrams with the same value function to be proved equal using its axioms.

The following proposition is the key technical lemma for the completeness proof in the last section. It states that two parametric linear programs with the same value function must share the same epigraph polyhedron.

Proposition 1.2. *Let*

$$v_i(y) := \inf_{x \in \mathbb{R}^{n_i}} \{c_i^\top x + d_i \mid A_i x \geq B_i y + e_i\}, \qquad i = 1, 2,$$

where $A_i \in \mathbb{R}^{m_i \times n_i}$, $B_i \in \mathbb{R}^{m_i \times k}$, $c_i \in \mathbb{R}^{n_i}$, $e_i \in \mathbb{R}^{m_i}$, *and* $d_i \in \mathbb{R}$. *If* $v_1(y) = v_2(y)$ *for all* $y \in \mathbb{R}^k$, *then both programs admit a common representation through the same epigraph polyhedron:*

$$v_i(y) = \inf\{t \mid (y, t) \in E\}, \qquad E := \mathrm{epi}(v_1) = \mathrm{epi}(v_2).$$

Proof. We use $\inf \varnothing = +\infty$ throughout. At a feasible parameter, a finite optimum is attained; if the objective is unbounded below, every real threshold t admits a feasible point with objective at most t. At an infeasible parameter, neither side of the following equivalence holds. Thus, for each $i = 1, 2$, the epigraph of v_i satisfies

$$(y, t) \in \mathrm{epi}(v_i) \iff \exists x : A_i x \geq B_i y + e_i,\ c_i^\top x + d_i \leq t,$$

so $\mathrm{epi}(v_i)$ is the projection onto (y, t) of the polyhedron $P_i = \{(x, y, t) \mid A_i x \geq B_i y + e_i,\ c_i^\top x + d_i \leq t\}$. Projections of polyhedra are polyhedra, so each $\mathrm{epi}(v_i)$ is a polyhedron. Since $v_1 = v_2$ pointwise, we have $\mathrm{epi}(v_1) = \mathrm{epi}(v_2) =: E$. Setting $E = \{(y, t) \mid Cy + at \geq h\}$ for suitable coefficients, we obtain $v_i(y) = \inf\{t \mid Cy + at \geq h\}$ for both $i = 1, 2$. □

2 COMPOSITION AND ORDER

This chapter makes the connection between composition, variable elimination, and optimization precise. We develop the categorical and order-theoretic structures that provide the foundation for the graphical language. Readers familiar with symmetric monoidal categories and quantales may use this chapter as a reference for our conventions.

2.1 RELATIONS AND QUANTALES

We begin with the most basic kind of relation:

Definition 2.1 (Binary Relations)**.** *Let X and Y be sets. A* binary relation *R from X to Y is a subset*

$$R \subseteq X \times Y.$$

Equivalently, it can be identified with its characteristic function

$$\chi_R : X \times Y \to \{0, +\infty\},$$

defined by

$$\chi_R(x, y) = \begin{cases} 0 \text{ if } (x, y) \in R, \\ +\infty \quad \text{otherwise.} \end{cases}$$

A binary relation specifies whether two elements are related. We use two representations: a subset of $X \times Y$ and its characteristic function, which assigns 0 to related pairs and $+\infty$ to unrelated pairs. Although Boolean characteristic functions conventionally use truth values or $\{0, 1\}$, the $\{0, +\infty\}$ convention anticipates the cost-valued relations introduced below.

The sets X and Y are arbitrary. Equipping them with additional structure allows us to consider more specific classes of relations.

Definition 2.2 (Linear/Affine Relations)**.** *Let X and Y be vector spaces. A* linear relation *from X to Y is a linear subspace*

$$R \subseteq X \times Y.$$

An affine relation *is an affine subspace of $X \times Y$.*

Equivalently, a linear relation can be described as the graph of a possibly multivalued linear operator, i.e.,

$$R(x) := \{y \in Y \mid (x, y) \in R\}.$$

Alternatively, its characteristic function is defined by

$$\chi_R : X \times Y \to \{0, +\infty\},$$

$$\chi_R(x,y) = \begin{cases} 0 \text{ if } (x,y) \in R, \\ +\infty \quad \text{otherwise.} \end{cases}$$

Thus, linear and affine relations are binary relations with additional geometric structure on the set of related pairs.

Binary relations record only whether a pair is related. To attach a cost or other numerical weight to each pair, we use the more general notion of *weighted relations* [Law73].

Definition 2.3 (Weighted Relations)**.** *Let X and Y be any two sets and let $\overline{\mathbb{R}}^+ := [0, +\infty]$. A* weighted relation *from X to Y is a set*

$$R \subseteq X \times Y \times \overline{\mathbb{R}}^+$$

such that for every $(x,y) \in X \times Y$ there exists exactly one *$r \in \overline{\mathbb{R}}^+$ satisfying*

$$(x,y,r) \in R.$$

Equivalently, we may define the characteristic function

$$\chi_R : X \times Y \to \overline{\mathbb{R}}^+$$

by

$$\chi_R(x,y) = r \quad \text{if and only if } (x,y,r) \in R.$$

The codomain is not restricted to a binary set anymore. In fact, we could choose any set of weights instead of $\overline{\mathbb{R}}^+$. But so far we have only described relations individually. We want more than that: we want to connect them, as we did with the optimization problems in the previous chapter.

Let us first consider ordinary binary relations $R \subseteq X \times Y$ and $S \subseteq Y \times Z$. If x is related to y by R, and y is related to z by S, then we can connect the two relations through this shared y. Their *composition* is the relation

$$S \circ R := \{(x,z) \in X \times Z \mid \exists y \in Y : \ (x,y) \in R \text{ and } (y,z) \in S\}.$$

Notice that y is no longer part of the resulting pair. We only ask whether some such y exists. This is precisely the idea of eliminating an inner variable while keeping the external ones.

Now, what happens when the relations carry weights? For each intermediate choice of y, we also need to determine the weight of the connection. If the weights represent costs, we add the costs of the two parts:

$$\chi_R(x,y) + \chi_S(y,z).$$

Different intermediate choices of y may give different costs. As in the optimization example of the previous chapter, we take the infimum over these choices:

$$\chi_{S\circ R}(x,z) := \inf_{y\in Y}\big(\chi_R(x,y) + \chi_S(y,z)\big).$$

Here $+\infty$ represents an unavailable connection, and we use $\inf\varnothing = +\infty$. If the weights are only 0 and $+\infty$, this formula gives 0 exactly when some intermediate choice satisfies both relations. Thus, it recovers ordinary relational composition.

So, choosing a set of weights is not enough to compose weighted relations, we also need an operation to combine weights along a connection and another to aggregate the possible intermediate choices. For costs, these are addition and infimum; other kinds of weights may call for other operations.

Furthermore, connecting three relations should give the same result regardless of how we group the connections, and inserting an identity relation should change nothing. To support this, we ask for an associative combination with a unit, arbitrary aggregation, and distributivity of combination over aggregation. These requirements lead us to another mathematical structure: a *quantale*.

Definition 2.4 (Quantale)**.** *A* quantale *is a structure* $(Q, \leq, \otimes, e)$ *such that:*

- $(Q, \leq)$ *is a complete lattice, i.e., every subset* $A \subseteq Q$ *admits a join* $\bigvee A$ *and a meet* $\bigwedge A$*;*
- $(Q, \otimes, e)$ *is a monoid;*
- *Multiplication distributes over arbitrary joins in each variable:*

$$a \otimes \Big(\bigvee_{i\in I} b_i\Big) = \bigvee_{i\in I}(a\otimes b_i), \qquad \Big(\bigvee_{i\in I} a_i\Big)\otimes b = \bigvee_{i\in I}(a_i\otimes b)$$

  for all families $\{a_i\}_{i\in I}, \{b_i\}_{i\in I} \subseteq Q$*.*

If $\otimes$ *is commutative, the quantale is called* commutative*. If* $e = \top$ *(the top element of the lattice), it is called* integral*.*

Each of these three axioms earns its place. The complete lattice gives meaning to the join $\bigvee$, which aggregates the possible intermediate choices. The monoid $(Q, \otimes, e)$ combines weights along a connection, with e the weight of doing nothing. Distributivity lets this combination pass through the aggregation, so that composing three relations gives the same result regardless of how we group them.

Notice that the definition uses joins, while our cost example used an infimum. For $\overline{\mathbb{R}}^{+} = [0, +\infty]$, we choose the reversed numerical order: $a \leq_Q b$ means $a \geq b$ numerically. The join in this order is precisely the numerical infimum. With addition as $\otimes$ and 0 as the unit, we

recover the operations used above. Thus, a quantale structures the weights in a way that supports composition; it is not itself a relation.

Given a quantale Q, we can define the notion of Q-valued relation [Coe+18].

Definition 2.5 (Generalized Relations)**.** *A* generalized relation *(or Q-valued relation) from X to Y, where Q is a quantale, is a set*

$$R \subseteq X \times Y \times Q$$

such that for every $(x, y) \in X \times Y$ there exists exactly one $q \in Q$ *satisfying*

$$(x, y, q) \in R.$$

Its characteristic function is

$$\chi_R : X \times Y \to Q$$

defined by

$$\chi_R(x, y) = q \quad \textit{if and only if } (x, y, q) \in R.$$

A Q-valued relation, or a Q-relation, is, in essence, a way to assign weights to pairs of elements. The quantale structure allows us to compose these relations: for each intermediate y, we combine the weights of the two connections, and then aggregate all possible intermediate choices. Thus, $(H \circ R)(x, z) = \bigvee_{y \in Y} R(x, y) \otimes H(y, z)$. For our cost quantale, this means adding the costs along each connection and taking the infimum over the intermediate choices: $(H \circ R)(x, z) = \inf_{y \in Y}\big(R(x, y) + H(y, z)\big)$.

One can recall our initial examples using this new structure:

Example 2.1. *The notion of a Q-valued relation recovers familiar concepts for particular choices of quantale.*

(1) Truth quantale. *Let $Q = \{0, 1\}$ with the usual order $0 \leq 1$, join given by $\vee$, and monoidal product $\otimes = \wedge$ with unit $e = 1$. This is the (Boolean) truth quantale.*

A Q-valued relation is then a function

$$R : X \times Y \to \{0, 1\}.$$

Such a function is precisely the characteristic function of an ordinary binary relation

$$\widetilde{R} \subseteq X \times Y, \qquad (x, y) \in \widetilde{R} \iff R(x, y) = 1.$$

Thus Q-valued relations coincide with classical binary relations.

This also settles the indicator convention adopted in Definition 2.1, where relations were valued in $\{0, +\infty\}$ rather than $\{0, 1\}$. The two are the same quantale: the map $1 \mapsto 0$, $0 \mapsto +\infty$ sends $\vee$ to $\min$, $\wedge$ to $+$, and the unit 1 to the unit 0, hence is an isomorphism of the truth quantale

onto the sub-quantale $\{0, +\infty\}$ *of the Lawvere quantale below. We work with* $\{0, +\infty\}$ *throughout, so that Boolean relations sit inside the cost-valued ones without any change of convention.*

(2) Lawvere quantale. *Let* $Q = \overline{\mathbb{R}}^{+} = [0, +\infty]$ *ordered by the* reverse *of the usual order (so that* $a \leq b$ *iff* $a \geq b$ *in the usual order), with monoidal product given by addition*

$$a \otimes b := a + b,$$

and unit $e = 0$*. Joins are then given by infima in the usual order.*

This structure is the Lawvere quantale. A Q-valued relation

$$R : X \times Y \to \overline{\mathbb{R}}^{+}$$

is precisely a weighted relation assigning a (possibly infinite) cost to each pair (x, y)*.*

Hence generalized relations unify ordinary binary relations and weighted relations as special cases corresponding to particular quantales.

Note that linear and affine relations are not recovered by this construction, since they demand extra structure on the sets X and Y, specifically that R be a linear or affine subspace of $X \times Y$. They are, nonetheless, fully embedded in the framework of binary relations.

To allow negative costs and unbounded objective values, we extend the Lawvere quantale to the entire extended real line:

Definition 2.6 (Optimization/Extended Lawvere Quantale)**.** *Let*

$$\overline{\mathbb{R}} := \mathbb{R} \cup \{-\infty, +\infty\}.$$

The optimization quantale *is the structure*

$$(\overline{\mathbb{R}}, \geq, \otimes, e)$$

defined as follows:

- *The order* $\geq$ *is the usual order on the extended real line. With this order, arbitrary joins are given by the usual infima:*
$$\bigvee A = \inf A.$$
- *The monoidal product is addition:*
$$a \otimes b := a + b,$$
where addition is extended to $\overline{\mathbb{R}}$ *by* $a + (+\infty) = +\infty$ *for every* a*, and* $a + (-\infty) = -\infty$

for every $a \neq +\infty$. In particular we adopt the convention

$$(+\infty) + (-\infty) := +\infty,$$

so that $+\infty$ is absorbing. This is Moreau's inf-addition *[Roc70], and it is the only extension under which the structure below is a quantale.*

- *The unit element is*

$$e := 0.$$

With these operations, $(\overline{\mathbb{R}}, \geq, +, 0)$ is a commutative quantale. Joins are ordinary infima, and addition distributes over arbitrary joins.

The convention above is not a matter of taste. Without it, $+$ is only partially defined on $\overline{\mathbb{R}}$, so $(\overline{\mathbb{R}}, +, 0)$ fails to be a monoid and the quantale axioms cannot even be stated. Following the pattern, we can build a Q-relation out of the previous quantale.

Definition 2.7 (Optimization (Cost/Reward) Relations)**.** *Let X and Y be sets. An* optimization relation *from X to Y is an $\overline{\mathbb{R}}$-valued relation, i.e., a function*

$$R : X \times Y \to \overline{\mathbb{R}}.$$

Equivalently, it is a subset

$$R \subseteq X \times Y \times \overline{\mathbb{R}}$$

such that for every $(x, y) \in X \times Y$ there exists exactly one $r \in \overline{\mathbb{R}}$ satisfying

$$(x, y, r) \in R.$$

The value $R(x, y)$ is interpreted as the objective value (cost or reward) associated to the pair (x, y), with $+\infty$ and $-\infty$ representing extremal infeasibility or unboundedness.

Optimization relations extend nonnegative weighted relations to negative values. The values $+\infty$ and $-\infty$ allow us to represent infeasibility and unboundedness below, respectively.

Quantale-valued relations can also be viewed as matrices. Under this identification, composition is matrix multiplication with joins and quantale multiplication in place of ordinary addition and multiplication.

Remark 2.1. *A Q-valued relation $R : X \times Y \to Q$ is nothing but a Q-*valued matrix *indexed by $X \times Y$: one simply reads off the entry at position (x, y) as $\chi_R(x, y) \in Q$. That is, a generalized relation is precisely an element of $Q^{X \times Y}$.*

When X and Y are finite sets, say $|X| = m$ and $|Y| = n$, this becomes completely explicit: a

Q-valued relation is exactly an $m \times n$ matrix

$$M_R \;=\; \big[\chi_R(x_i,\, y_j)\big]_{\substack{1\le i\le m\\ 1\le j\le n}} \;\in\; Q^{m\times n},$$

whose (i, j)-entry records the Q-value assigned to the pair (x_i, y_j).

Moreover, composition of Q-valued relations corresponds precisely to *matrix multiplication in* Q. Given a second Q-valued relation $S : Y \times Z \to Q$, their composite $S \circ R : X \times Z \to Q$ is defined by

$$\chi_{S\circ R}(x, z) \;=\; \bigvee_{y\in Y} \chi_R(x, y) \otimes \chi_S(y, z),$$

which, in matrix notation, reads

$$M_{S\circ R} \;=\; M_R \otimes M_S, \qquad (M_R \otimes M_S)_{ij} \;=\; \bigvee_{k=1}^{n} (M_R)_{ik} \otimes (M_S)_{kj}.$$

This is the usual row-times-column pattern, with ordinary addition replaced by the join $\bigvee$ and ordinary multiplication replaced by $\otimes$. Associativity of $\otimes$ and its distributivity over arbitrary joins make this matrix composition associative.

For cost-valued relations, the same construction adds costs along a connection and takes the infimum over intermediate choices. This is the composition used for linear programs in the preceding chapter.

2.2 CATEGORIES AND SEMANTICS

These constructions expose the order-theoretic structure of relational composition: weights are combined along each intermediate connection and then aggregated using a join [FS19a, Section 2.5.3]. For costs, this join is an infimum. The resulting value need not be attained by any intermediate choice.

Linear programs and cost-valued relations therefore share a compositional operation. Category theory provides a framework for making this connection precise. As John Baez writes [Bae04, Section 2, p. 6], "every sufficiently good analogy is yearning to become a functor".

Readers unfamiliar with categories and functors may consult Appendix **??**, particularly Section **??**.

We first construct a category of Q-valued relations for any quantale Q. To describe independent systems in parallel, we also need a tensor product that combines their interfaces and morphisms.

Definition 2.8 (Symmetric monoidal category)**.** *A* symmetric monoidal category *is a category* $\mathcal{C}$ *with a tensor product* $\otimes$*, a unit object I, and coherent natural isomorphisms for associativity, units, and symmetry. For* $f : A \to B$ *and* $g : C \to D$*, the tensor gives*

$$f \otimes g : A \otimes C \longrightarrow B \otimes D.$$

It preserves identities and satisfies the interchange law:

$$(f_2 \circ f_1) \otimes (g_2 \circ g_1) = (f_2 \otimes g_2) \circ (f_1 \otimes g_1).$$

The symmetry $\sigma_{A,B} : A \otimes B \to B \otimes A$ exchanges the two interfaces. The unit I represents an empty interface.

In other words, composing independent systems separately or first placing them in parallel gives the same result. We suppress the canonical associativity and unit isomorphisms in our notation; this does not mean they are identities in every category. The coherence conventions are discussed in Appendix **??**.

The following proposition constructs the category of Q-valued relations. When Q is commutative, this category is symmetric monoidal and compact closed. Compact closure is defined in Appendix **??**. The construction is described in [Coe+18]:

Proposition 2.1 (Category $\mathbf{Rel}(Q)$)**.** *For a quantale Q, the Q-relations form a category **Rel**(Q) with composition and identities*

$$(S \circ R)(a,c) = \bigvee_b R(a,b) \otimes S(b,c) \quad 1_A(a,b) = \bigvee \{k | a = b\}$$

*If Q is a commutative quantale, then **Rel**(Q) carries a symmetric monoidal structure, with the categorical monoidal product $\bigotimes_{\mathcal{C}}$ on objects given by the Cartesian product of sets and the action on relations given for $R : A \to C$ and $S : B \to D$ by*

$$(R \otimes_{\mathcal{C}} S)(a,b,c,d) = R(a,c) \otimes S(b,d)$$

The singleton set is the monoidal unit, and the category is compact closed with respect to this monoidal structure.

Proof. We verify the categorical structure.

(1) Well-defined composition. Let $R : A \to B$ and $S : B \to C$ be Q-relations. For each $(a,c) \in A \times C$, the set

$$\{R(a,b) \otimes S(b,c) \mid b \in B\} \subseteq Q$$

admits a supremum since Q is a complete lattice. Hence $(S \circ R)(a,c)$ is well-defined as an element of Q.

(2) Associativity. Let $R : A \to B$, $S : B \to C$, and $T : C \to D$. For $a \in A$, $d \in D$,

$$((T \circ S) \circ R)(a,d) = \bigvee_c \Big(\bigvee_b R(a,b) \otimes S(b,c) \Big) \otimes T(c,d).$$

Since $\otimes$ distributes over arbitrary suprema,

$$= \bigvee_{b,c} R(a,b) \otimes S(b,c) \otimes T(c,d).$$

Similarly,

$$(T \circ (S \circ R))(a,d) = \bigvee_{b} R(a,b) \otimes \Big(\bigvee_{c} S(b,c) \otimes T(c,d) \Big) = \bigvee_{b,c} R(a,b) \otimes S(b,c) \otimes T(c,d).$$

Thus composition is associative.

(3) Identities. For each set A, define

$$1_A(a,b) = \begin{cases} e & \text{if } a = b, \\ \bot & \text{otherwise,} \end{cases}$$

where $\bot = \bigvee \varnothing$ is the bottom element of Q.

Then for $R : A \to B$,

$$(R \circ 1_A)(a,b) = \bigvee_{a'} 1_A(a,a') \otimes R(a',b).$$

All terms vanish except when $a' = a$, so

$$= e \otimes R(a,b) = R(a,b).$$

Similarly, $1_B \circ R = R$. Hence identities exist.

(4) Monoidal structure (commutative case). Assume Q is commutative. Define the tensor on objects by Cartesian product:

$$A \otimes_{\mathcal{C}} B := A \times B.$$

For relations $R : A \to C$ and $S : B \to D$, define

$$(R \otimes_{\mathcal{C}} S)((a,b),(c,d)) = R(a,c) \otimes S(b,d).$$

Functoriality follows from associativity and commutativity of $\otimes$:

$$(S_1 \otimes S_2) \circ (R_1 \otimes R_2) = (S_1 \circ R_1) \otimes (S_2 \circ R_2).$$

The singleton set 1 is a unit since $A \times 1 \cong A$.

Symmetry is induced by the swap map

$$\sigma_{A,B}((a,b),(b',a')) = \begin{cases} e & \text{if } a = a',\ b = b', \\ \bot & \text{otherwise.} \end{cases}$$

(5) Compact closure. For each set A, define unit and counit relations

$$\eta : 1 \to A \times A, \qquad \epsilon : A \times A \to 1$$

by

$$\eta(*,(a,a')) = \begin{cases} e & a = a', \\ \bot & \text{otherwise,} \end{cases} \qquad \epsilon((a,a'),*) = \begin{cases} e & a = a', \\ \bot & \text{otherwise.} \end{cases}$$

A direct computation using the definition of composition shows that the triangle identities hold:

$$(1_A \otimes \epsilon) \circ (\eta \otimes 1_A) = 1_A, \qquad (\epsilon \otimes 1_A) \circ (1_A \otimes \eta) = 1_A.$$

Thus every object is self-dual, and $\mathbf{Rel}(Q)$ is compact closed. □

Example 2.2 (Category of weighted relations)**.** *Let*

$$Q = ([0,+\infty], \leq_Q, \otimes, e)$$

be the nonnegative Lawvere quantale [Law73; FS19a] where:

- $a \leq_Q b \iff a \geq b$ *in the usual order;*
- $\otimes = +$*;*
- $e = 0$*;*
- *joins are given by infima in the usual order.*

A Q-valued relation $R : A \to B$ is a function

$$R : A \times B \to [0,+\infty],$$

assigning a cost (or weight) to each pair (a,b).

The composition formula becomes

$$(S \circ R)(a,c) = \inf_{b \in B} \big(R(a,b) + S(b,c)\big).$$

Thus composition is given by infimal convolution *[SS24].*

Since addition is commutative, the Lawvere quantale is commutative, and $\mathbf{Rel}(Q)$ carries a symmetric monoidal structure:

- *On objects:*

$$A \otimes B := A \times B.$$

- *On morphisms:*

$$(R \otimes S)((a,b),(c,d)) = R(a,c) + S(b,d).$$

The singleton set is the monoidal unit. With this tensor product, $\mathbf{Rel}(Q)$ *is a symmetric monoidal compact closed category of weighted relations.*

In particular:

- *If* A, B, C *are finite sets, relations may be viewed as matrices with entries in* $[0, +\infty]$.
- *Composition is matrix multiplication in the* $(\min, +)$*-semiring.*

Hence $\mathbf{Rel}(Q)$ *is a category of nonnegative weighted relations, which underlies dynamic programming, tropical algebra, and shortest-path problems. For costs that also admit negative values and* $-\infty$*, we instead use the optimization quantale defined above, with its inf-addition convention.*

This example illustrates how the quantale structure supports categorical constructions: composition of Q-valued relations corresponds to matrix multiplication driven by $\bigvee$, and the tensor product of relations is governed by the multiplication $\otimes$, both operations coming directly from the quantale axioms.

These constructions provide exactly the semantic foundation for the algebra we aim to build. In the same way that semantics is understood in linguistics and programming language theory [Win93], the semantics of a formal system assigns meaning to its expressions. More precisely, while syntax consists of the tokens and rules for forming and combining expressions, semantics is the interpretation assigned to each such expression — in our case, a cost-valued relation living in $\mathbf{Rel}(Q)$.

As outlined in the introduction, the central goal of this work is to show that optimization is merely a particular case of a more general algebraic and categorical structure. The key mechanism enabling this is the infimal composition present in the definition of the Costs/Rewards category: composing two relations automatically performs variable elimination, which is precisely the fundamental operation underlying optimization.

To make this precise, we need both a semantics — provided by the categorical framework developed above — and a syntax: a rigorous, compositional language for expressing optimization problems symbolically. The map between those is a structure-preserving map, a symmetric monoidal functor, from the syntactic layer into the semantic one.

Definition 2.9 (Symmetric monoidal functor)**.** *A* strong symmetric monoidal functor $F : \mathcal{C} \to \mathcal{D}$ *is a functor with coherent natural isomorphisms*

$$FA \otimes FB \longrightarrow F(A \otimes B), \qquad I_{\mathcal{D}} \longrightarrow F(I_{\mathcal{C}}),$$

compatible with associativity, units, and symmetry. When these comparisons are identities, the functor is called strict.

For our interpretations, this preservation is written as

$$[\![g \circ f]\!] = [\![g]\!] \circ [\![f]\!], \qquad [\![f \otimes g]\!] = [\![f]\!] \otimes [\![g]\!].$$

where $[\![\,]\!]$ is our interpretation map. This map takes a symbol or expression in our syntax and maps it to its meaning. For the purpose here, the interpretation map will take diagrams (the symbols of our syntax) and map them to LP value functions, represented as cost-valued relations. Identities, the unit, and symmetry are preserved as well. Thus, interpreting a compound diagram gives the same result as interpreting its components and then combining their meanings. For a general strong functor, the corresponding identifications use its comparison isomorphisms rather than literal equalities. The formal compatibility conditions are recalled in Appendix **??** and the definition of the map is presented later in this section.

The graphical calculi used in this thesis have one generating wire type. An interface is therefore determined only by its number of wires.

Definition 2.10 (PROP)**.** *A* PROP *is a strict symmetric monoidal category whose objects are the natural numbers, whose tensor product on objects is addition, and whose monoidal unit is* 0*. A morphism $m \to n$ is drawn with m input wires and n output wires.*

Thus 1 is the basic wire, $n = 1^{\otimes n}$ is a bundle of n wires, and $m \otimes n = m + n$. Matrices provide the standard example: for a field $\mathbb{K}$, let $\mathbf{Mat}_{\mathbb{K}}$ have morphisms $m \to n$ given by $n \times m$ matrices. Composition is matrix multiplication, tensor is block diagonal sum, and the symmetry is a permutation matrix.

A morphism of PROPs is a strict symmetric monoidal functor that is the identity on objects. It preserves the number of inputs and outputs, sequential composition, parallel composition, identities, and symmetries [BSZ17, Section 2.1].

The syntactic layer will be formalized as a Symmetric Monoidal Theory (SMT) [BSZ17], which is a presentation of a category by generators and equations, whose terms respect the compositional structure of a symmetric monoidal category. This is the standard tool for building graphical languages, since it provides the necessary structures to allow for parallel and sequential composition.

Definition 2.11 (Symmetric Monoidal Theory)**.** *An SMT is a pair (Σ, E), where Σ is a signature of operators, each with a specified arity m and coarity n, and E is a set of equations.*

Σ-terms are defined inductively as the smallest collection of typed expressions $t : m \to n$ such that:

1. *(Generators) If $o \in \Sigma$ with $o : m \to n$, then o is a Σ-term of type $m \to n$.*

2. *(Identity) For every $n \in \mathbb{N}$, there is a term*

$$\mathrm{id}_n : n \to n.$$

3. *(Composition) If*

$$a : m \to n \quad \textit{and} \quad b : n \to k$$

are Σ-terms, then

$$a; b : m \to k$$

is a Σ-term.

4. *(Tensor product) If*

$$a : m_1 \to n_1 \quad \textit{and} \quad b : m_2 \to n_2$$

are Σ-terms, then

$$a \otimes b : m_1 + m_2 \to n_1 + n_2$$

is a Σ-term.

5. *(Symmetry) For all $m, n \in \mathbb{N}$, there is a term*

$$\sigma_{m,n} : m + n \to n + m.$$

The equations in E relate Σ-terms of the same type.

We represent generators $o : m \to n$ in Σ as string diagrams. The Σ-terms are built inductively by sequential and parallel composition of these generators and the structural terms. The set E supplies the axiomatic equations between diagrams.

The categorical counterpart of these terms is the free category generated by the SMT.

Definition 2.12 (The Free category of SMT)**.** *The category $\mathrm{Free}P_{(\Sigma,E)}$ is freely generated by (Σ, E) where the objects are natural numbers, morphisms are Σ-terms modulo the symmetric monoidal structural laws and the smallest congruence generated by E. This congruence is closed under composition and tensor product, so these operations are well-defined on equivalence classes. $\mathrm{Free}P_{(\Sigma,E)}$ is in fact a PROP*[1]

The $\mathrm{Free}P_{(\Sigma,E)}$, also denoted as $P\Sigma_{/E}$, [2] will be our **syntax** responsible for expressing our underlying semantics.

An SMT supports equational reasoning: proofs apply equations in E within larger terms, together with the symmetric monoidal structural laws. Sequential and parallel composition provide the contexts in which these local substitutions are made.

[1] For the definition of a PROP, see Definition 2.10; its presentation by an SMT is recalled in Appendix **??**.

[2] Throughout this work, we use $\mathrm{Free}P_S$ and S, for whatever syntax S may be, interchangeably.

This resembles formal reasoning about computer programs, which may also be composed sequentially or in parallel [Win93; PZ23]. Here, the components are linear programs, and the aim is to reason about their compositions using equations in a domain-specific language. To interpret them, we need a map from those symbols into objects with more expressive meaning.

Definition 2.13 (Interpretation)**.** *The interpretation* $[\![-]\!] : \mathrm{FreeP}_{\mathbf{P}_{/\mathbf{E}}} \to \mathcal{C}$ *is the strict symmetric monoidal functor that sends each generator of* $\mathbf{P}$ *to its semmantics, read as a object in the category* $\mathcal{C}$*.*

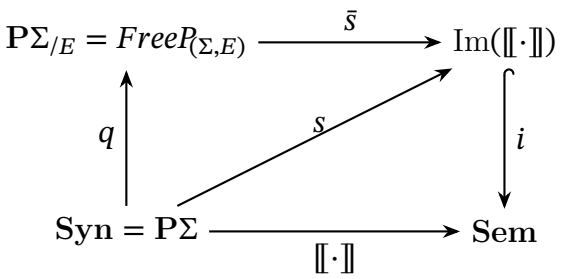
$\mathcal{C}$ here is read as the semantics, that is, the objects and structures in which we are interested. A syntax is then another category whose exists a functor $[\![]\!]$ which maps their objects in the semantics in a suitable manner. One can argue to which extent one is really *semantics* and the other *syntax*.

We often consider the *syntax* the objects we are interested in using to write down the proofs and our reasoning, we can then impose over it suitable properties, such as those from SMTs, making their use very convenient. In the other hand, semantics are often the objects onto one wants to understand facts and propositions about it.

One could say that the semantics objects are "more close related" to the actual objects in which it thinks about. However, the truth is that when one writes $[\![z]\!] = f(x, y)$ it is just mapping symbols into symbols and the real meaning comes from the very act of reading it. We abstain from such philosophical dispute, here those terms are differentiated by the way we intent to use them and we will denote as *semantics* the traditional mathematical representation of the objects of interest.

To connect these proofs and equations to the intended semantics, we require soundness, completeness, and expressivity. These properties specify which semantic statements the calculus can establish and which relations it can represent.

Definition 2.14. *We say that an SMT axiomatizes or represents a specific semantic PROP when its interpretation is sound, complete, and expressive. Soundness gives the commuting diagram below; completeness and expressivity give the further properties described afterward.*



Here $\mathrm{P}\Sigma$ *is the free PROP of* Σ*-terms modulo the symmetric monoidal structural laws;* $\mathrm{P}\Sigma_{/E} = FreeP_{(\Sigma,E)}$ *is its quotient by the congruence generated by E; and* $q : \mathrm{P}\Sigma \to \mathrm{P}\Sigma_{/E}$ *witnesses this quotient.* $\mathbf{Sem}$ *is our intended semantic PROP, with natural numbers indexing*

the chosen interfaces, such as $\mathbb{R}^n$, rather than all the objects of $\mathbf{Rel}(Q)$. The map s is the interpretation of raw terms restricted to its semantic image; it need not be faithful. The arrow $\bar{s}$ exists when this interpretation is independent of the representative of an equivalence class. We distinguish three properties:

- *Soundness: We say that the interpretation is sound if $c \overset{E}{=} d$[3] implies $[\![c]\!] = [\![d]\!]$. This is exactly what makes the induced interpretation on $P\Sigma_{/E}$ well-defined.*
- *Completeness: We say that the interpretation is complete if $[\![c]\!] = [\![d]\!]$ implies $c \overset{E}{=} d$. Once soundness holds, the induced interpretation is then faithful.*
- *Expressivity: We say that the interpretation is expressive if for every morphism $f : n \to m$ in $\mathbf{Sem}$ there exists a term $d : n \to m$ such that $[\![d]\!] = f$. Once soundness holds, the induced interpretation into $\mathbf{Sem}$ is then full.*

Soundness and completeness make $\bar{s}$ an isomorphism onto the semantic image. Expressivity makes that image the whole intended semantic PROP. These statements concern the chosen interfaces and their hom-sets [BSZ17, Section 2.1]; see also Appendix **??**.

Those three properties capture the idea that everything we can prove in the syntax holds in the semantics (soundness), every equality that holds in the semantics is provable in the syntax (completeness) and every morphism in the semantics has a syntactic representative (expressivity). Moreover, these properties describe the interpretation map both categorically and functionally. In the table below, injectivity and surjectivity are understood separately on each hom-set of the induced quotient interpretation:

Table 2.1 – Equivalence of properties for the interpretation map $[\![-]\!] : P\Sigma_{/E} \to \mathbf{Sem}$

Set theory	**Category theory**	**Logic (SMT)**
Deterministic	Factors through the quotient	Soundness: $c \overset{E}{=} d \Rightarrow [\![c]\!] = [\![d]\!]$
Injective	Faithful	Completeness: $[\![c]\!] = [\![d]\!] \Rightarrow c \overset{E}{=} d$
Surjective	Full	Expressivity: each $f : n \to m$ in $\mathbf{Sem}$ has a term $d : n \to m$ with $[\![d]\!] = f$

[3] $\overset{E}{=}$ means that this equality is provable using E and the symmetric monoidal structural laws.

3 STRING DIAGRAMS AND GRAPHICAL AFFINE ALGEBRA

Having established the categorical background, we now introduce string diagrams and the calculi of Graphical Linear Algebra and Graphical Affine Algebra [Bon+19]. Readers familiar with these calculi may use this chapter as a reference for the notation and conventions adopted here.

3.1 STRING DIAGRAMS

A symmetric monoidal category has a sound two-dimensional notation: its *string diagrams* [JS91; Sel10]. A wire represents an object and a box with input and output wires represents a morphism. Reading from left to right, joining the outputs of f to the inputs of g represents $g \circ f$; placing f above g represents $f \otimes g$. A bare wire is an identity and crossing wires represents symmetry. The interchange law is built into the geometry: disjoint boxes may move past one another because their relative horizontal order carries no information. Consequently, diagrams are considered equal up to deformations that preserve connectivity, boundary, and the order of ports on each box. The basic translation between categorical notation and string diagrams is displayed in Figure 3.1.

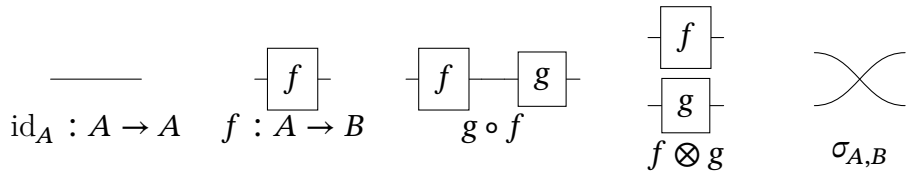
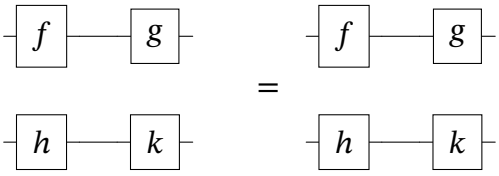


Figure 3.1 – String-diagram notation for the structure of a symmetric monoidal category. Diagrams are read from left to right: horizontal connection is composition, while vertical juxtaposition is the tensor product.

The labels beneath the pictures record the corresponding morphisms. In particular, the two wires in the drawing of $f \otimes g$ do not interact: they represent processes performed in parallel. The crossing represents the canonical reordering $A \otimes B \to B \otimes A$, not an additional operation chosen for the particular objects.



$(g \circ f) \otimes (k \circ h)$ $\qquad$ $(g \otimes k) \circ (f \otimes h)$

Figure 3.2 – The interchange law expressed as a string diagram.

For $\mathbf{Rel}$ the monoidal product is Cartesian product on objects. If $R : X \to Y$ and $S : U \to V$, then

$$R \otimes S : X \times U \longrightarrow Y \times V, \qquad ((x,u),(y,v)) \in R \otimes S \iff xRy \text{ and } uSv.$$

The singleton set is the unit. Sequential composition eliminates shared variables; monoidal composition juxtaposes independent systems.

String diagrams encode structural bookkeeping, such as the interchange law and the associativity of composition, in their geometry. This allows derivations to focus on the operations and constraints of the system rather than repeatedly invoking structural identities.

Diagrammatic equations can also be applied locally: a subdiagram may be replaced by another with the same boundary without changing the surrounding diagram. For the optimization language developed later, sequential composition denotes the infimum over shared variables. The notation therefore allows components to be manipulated without repeatedly expanding the corresponding infima.

3.2 FROBENIUS STRUCTURE AND HYPERGRAPH CATEGORIES

The resulting categories are symmetric monoidal, and it is precisely this monoidal backbone that will provide the algebraic structure in which we are interested. A key piece of that structure is a *Frobenius monoid* on each object (see Appendix **??**): a monoid (μ, η) and a comonoid (δ, ε) on the same object, satisfying the Frobenius law

$$(\mathrm{id} \otimes \mu) \circ (\delta \otimes \mathrm{id}) \;=\; \delta \circ \mu \;=\; (\mu \otimes \mathrm{id}) \circ (\mathrm{id} \otimes \delta).$$

For a hypergraph category, this structure must also be *commutative* and *special*: the multiplication and comultiplication respect the symmetry, and $\mu \circ \delta = \mathrm{id}$. A symmetric monoidal category with a chosen special commutative Frobenius monoid on every object, compatibly with the tensor product and its unit, is called a *hypergraph category* [FS19b, Definition 2.12].

Hypergraph categories carry a self-dual compact closed structure [FS19b, Proposition 3.1]. For each object A, its Frobenius operations define a cup and a cap by

$$\mathrm{cup}_A := \delta_A \circ \eta_A : I \to A \otimes A, \qquad \mathrm{cap}_A := \varepsilon_A \circ \mu_A : A \otimes A \to I.$$

These satisfy the snake equations, so A can serve as its own dual. It is this richer structure that we shall adopt as our ambient framework.

Remark 3.1. *If Q is commutative, then $\mathbf{Rel}(Q)$ has a canonical special commutative Frobenius structure on every object: its maps copy, discard, and take the relational converses of those operations, with weight e on allowed pairs and $\bot$ otherwise [Coe+18].*

This structure will justify identities in the calculi developed below. To illustrate its operations, we introduce the following symbols.

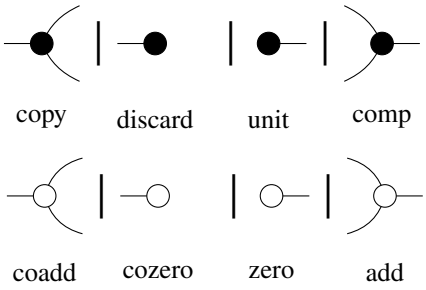
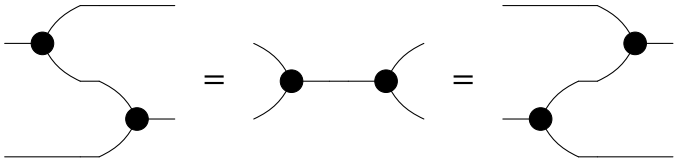


Figure 3.3 – Frobenius additive(black) and co-additive(white) structure

The black structure consists of copying and discarding operations and their relational converses. These converses enforce equality between two values or create an arbitrary value, respectively. The copying and additive structures on linear relations are described in [FS19b, Examples 2.10–2.11]. The Frobenius law gives the identities below. Copying the first value and comparing it with the second is equivalent to copying the second and comparing it with the first, or to comparing both values and then copying the common value.



3.3 GRAPHICAL LINEAR ALGEBRA

We now introduce an equational, diagrammatic syntax for linear algebra. We recall the relevant results without reproducing their proofs. The linear presentation and its proofs are given in [BSZ17], and the affine extension is developed in [Bon+19]. The categorical and graphical background is developed in Appendix **??** and Section 3.1.

Fix a field $\mathbb{K}$; all applications in this thesis use $\mathbb{K} = \mathbb{R}$.

Definition 3.1 (Linear relation)**.** *A* linear relation $R : m \to n$ *is a linear subspace*

$$R \subseteq \mathbb{K}^m \oplus \mathbb{K}^n.$$

We write $x\ R\ y$ *when* $(x, y) \in R$*. The PROP* $\mathbf{LinRel}_{\mathbb{K}}$ *has natural numbers as objects and linear*

relations as morphisms. Its identity, composition, and tensor are

$$\begin{aligned} \mathrm{id}_n &= \{(x, x) \mid x \in \mathbb{K}^n\}, \\ S \circ R &= \{(x, z) \mid \exists y \in \mathbb{K}^n \,:\, x\ R\ y \text{ and } y\ S\ z\}, \\ R \otimes S &= \{((x, u), (y, v)) \mid x\ R\ y,\ u\ S\ v\}. \end{aligned}$$

After the canonical identification $\mathbb{K}^m \oplus \mathbb{K}^p \cong \mathbb{K}^{m+p}$, the tensor on objects is addition.

The composite is again a linear subspace: the simultaneous constraints form a homogeneous linear system, and projecting away the intermediate variables preserves linearity. [1]

Every matrix $A \in \mathbb{K}^{n \times m}$ determines the functional linear relation

$$\mathrm{graph}(A) = \{(x, y) \in \mathbb{K}^m \oplus \mathbb{K}^n \mid y = Ax\}.$$

Connecting these functional relations recovers ordinary matrix multiplication over $\mathbb{K}$:

$$\mathrm{graph}(B) \circ \mathrm{graph}(A) = \mathrm{graph}(BA).$$

Graphical Linear Algebra is a graphical presentation of $\mathbf{LinRel}_{\mathbb{K}}$ [BSZ17]. Its diagrams are generated by two interacting families of nodes and one scalar box for each $k \in \mathbb{K}$. Reflecting a diagram exchanges its input and output interfaces. If the original diagram denotes $R : m \to n$, its reflection denotes the converse relation $R^{\mathrm{op}} : n \to m$, defined by

$$x\ R\ y \iff y\ R^{\mathrm{op}}\ x.$$

Reflection is a contravariant involution: it reverses the order of composition, and reflecting twice recovers the original relation. In general, R and R^{op} are different typed morphisms; reflection does not assert that they are equal or that R is invertible.

The following table gives the type and exact relation denoted by every basic orientation; the reversed orientation denotes the converse relation.

[1] Composition is relational composition, computed by projecting the simultaneous linear constraints onto the external variables.

Table 3.1 – Generators of Graphical Linear Algebra and their linear-relation semantics.

diagram	type	interpretation
	$2 \to 1$	$\{((x_1, x_2), y) \mid y = x_1 + x_2\}$
	$0 \to 1$	$\{(*, 0)\}$
	$1 \to 2$	$\{(x, (y_1, y_2)) \mid y_1 = x = y_2\}$
	$1 \to 0$	$\{(x, *) \mid x \in \mathbb{K}\}$
	$1 \to 1$	$\{(x, y) \mid y = kx\}$, for $k \in \mathbb{K}$

The white node is addition and the black node is equality. Their reflected forms have the following readings:

- white comultiplication $1 \to 2$ imposes $x = y_1 + y_2$;
- white counit $1 \to 0$ imposes $x = 0$;
- black multiplication $2 \to 1$ imposes $x_1 = x_2 = y$;
- black unit $0 \to 1$ allows an arbitrary value of y;
- a reflected scalar imposes $x = ky$.

Thus direction distinguishes solving an equation forward from treating the same equation as a constraint. When $k \neq 0$, a reflected k-box is equivalent to a forward k^{-1}-box; when $k = 0$, its semantics is the non-functional constraint $x = 0$ with arbitrary output.

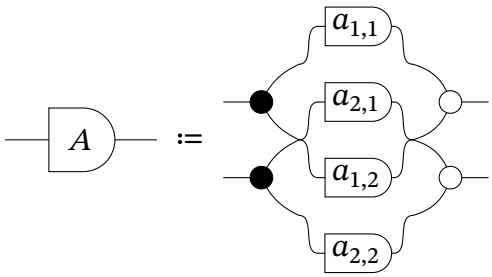
These generators allow us to represent an arbitrary matrix:

Definition 3.2 (Matrix diagram)**.** *Let $A = (a_{ij}) \in \mathbb{K}^{n \times m}$. The matrix diagram $A : m \to n$ is built from the generators above as follows. Copy each input x_j into n wires, one for each output. On the wire from input j to output i, place the scalar box a_{ij}. Reorder the wires to group them by output, and add the m contributions in each group. Thus output i is*

$$y_i = \sum_{j=1}^{m} a_{ij} x_j,$$

and the diagram denotes exactly $\mathrm{graph}(A)$.

The following example illustrates the construction for a 2×2 *matrix; the same procedure applies to an* $n \times m$ *matrix.*



The equational theory of Graphical Linear Algebra can be organized into four layers.

(i) The addition node and zero form a commutative monoid, while copy and discard form a cocommutative comonoid. Reflecting these equations gives the corresponding structures in the opposite direction.

(ii) Each color, together with its reflection, is a special commutative Frobenius algebra. The spider theorem therefore applies separately to connected monochromatic regions.

(iii) The two colors interact by the bialgebra and Hopf laws. These laws express that copying preserves addition and zero, and that additive inverses cancel. The scalar -1 is the antipode.

(iv) Scalar boxes obey the arithmetic of $\mathbb{K}$: consecutive boxes multiply, parallel copied-and-added boxes add, 1 is the identity, and scalars commute with the appropriate copying and addition nodes.

Every equation is sound because both sides denote the same linear subspace. For example, copying after addition and adding copied components both denote

$$\{((x_1, x_2), (y_1, y_2)) \mid y_1 = x_1 + x_2 = y_2\}.$$

The equations do more than preserve semantics: they characterize it.

Definition 3.3 (The linear syntax)**.** *We call* $\mathbf{Graphical\ Linear\ Algebra}_{\mathbb{K}}$ *the PROP built from the linear generators and equations presented above [BSZ17]. A diagram* $m \to n$ *has* m *input wires and* n *output wires. We build diagrams by connecting these generators or placing them in parallel, and use the equations to identify them, as in our definition of a graphical theory. The affine point and its axioms are not included.*

Notice that we already have a language here. We can express $x + y = 0$, but not $x + y = 1$. Why? Because every linear relation admits the zero assignment, and the second equation does not. The theorem below tells us that this smaller language captures exactly $\mathbf{LinRel}_{\mathbb{K}}$. We will start from this same language when we add inequalities in Chapter 4.

Theorem 3.1 (Presentation theorem for Graphical Linear Algebra)**.** *For a field* $\mathbb{K}$*, the standard interpretation extends to an isomorphism of PROPs*

$$[\![-]\!] : \mathbf{Graphical\ Linear\ Algebra}_{\mathbb{K}} \xrightarrow{\cong} \mathbf{LinRel}_{\mathbb{K}}.$$

Consequently, Graphical Linear Algebra is sound, complete, and expressive for linear relations [BSZ17].

Linear relations are more general than linear maps. A reflected matrix denotes the converse relation

$$\mathrm{graph}(A)^{\mathrm{op}} = \{(y, x) \mid y = Ax\},$$

which may be partial or multivalued. Every linear relation has a homogeneous cospan form $Ex = Fy$, and hence a diagrammatic representative.

A matrix box $A : m \to n$ abbreviates any Graphical Linear Algebra diagram whose interpretation is $y = Ax$. Its reflected box denotes the converse relation. For compatible matrices, connecting boxes gives matrix multiplication, while copying inputs, applying two matrices, and adding their outputs gives the corresponding block matrix. A general linear relation box may be replaced by its homogeneous cospan form

$$x \; R \; y \quad \Longleftrightarrow \quad Ex = Fy.$$

3.4 GRAPHICAL AFFINE ALGEBRA

Without the affine point, all generators denote homogeneous linear relations. We now extend Graphical Linear Algebra with the affine point $\vdash : 0 \to 1$, which denotes the constant 1. Together with scalar multiplication and copying, it constructs constants and translations. The resulting language is Graphical Affine Algebra [Bon+19].

$$[\![\vdash]\!] = \{(*, 1)\}$$

Definition 3.4 (The affine syntax)**.** *We call* $\mathbf{Graphical\ Affine\ Algebra}_{\mathbb{K}}$ *the PROP built by adding the affine point and its three equations to the linear syntax of Definition 3.3. We build and identify diagrams in the same way as before, now using both the linear and affine equations.*

What changes is that we can now supply the constant 1. Multiplying it by a scalar gives any constant we need, and adding it to a variable lets us write equations such as $x + y = 1$. More generally, $Ex = Fy$ becomes $Ex + b = Fy$. We have not changed how diagrams connect; we have added one component and the equations that tell us how to use it. If we leave that component out, we are back to diagrams describing linear relations.

The same distinction will matter in the next chapter. Adding inequalities to the smaller linear language gives polyhedral cones. Adding them to the affine language gives general polyhedra. Once again, allowing constants is what makes the difference.

This extension illustrates the incremental construction of the language: a new generator and its equations add a specific semantic capability. The same approach will be used to introduce inequalities and objectives, extending the calculus to polyhedra and linear programs. Comparing these presentations identifies the additional operations used to pass from one class of relations to another.

Definition 3.5 (Affine relation)**.** *An* affine relation $R : m \to n$ *is an affine subspace (or the empty set)*

$$R \subseteq \mathbb{K}^m \oplus \mathbb{K}^n.$$

We write $x\ R\ y$ *when* $(x, y) \in R$*. The PROP* $\mathbf{AffRel}_{\mathbb{K}}$ *has natural numbers as objects and affine relations as morphisms. Its identity, composition, and tensor are*

$$\begin{aligned} \mathrm{id}_n &= \{(x, x) \mid x \in \mathbb{K}^n\}, \\ S \circ R &= \{(x, z) \mid \exists y \in \mathbb{K}^n : x\ R\ y \text{ and } y\ S\ z\}, \\ R \otimes S &= \{((x, u), (y, v)) \mid x\ R\ y,\ u\ S\ v\}. \end{aligned}$$

After the canonical identification $\mathbb{K}^m \oplus \mathbb{K}^p \cong \mathbb{K}^{m+p}$*, the tensor on objects is addition.*

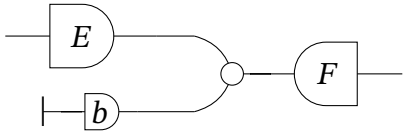
The composite is again an affine subspace or empty: the simultaneous constraints $(x, y) \in R$ and $(y, z) \in S$ form an affine system, and projecting away y preserves affinity. Hence composition of affine relations is exactly affine variable elimination.

Affine relations are nevertheless strictly more general than affine maps. Linear relations embed as the special case with zero translation. More generally, every affine relation admits a presentation by affine linear equations:

$$R = \{(x, y) \mid Ex + b = Fy\}$$

for suitable matrices E, F and vector b. This affine cospan form is the semantic origin of Graphical Affine Algebra normal forms.

Remark 3.2. *Every morphism in* ***AffRel*** *admits the following diagram representation*



Theorem 3.2 (Presentation theorem for Graphical Affine Algebra)**.** *For a field* $\mathbb{K}$*, the standard interpretation extends to an isomorphism of PROPs*

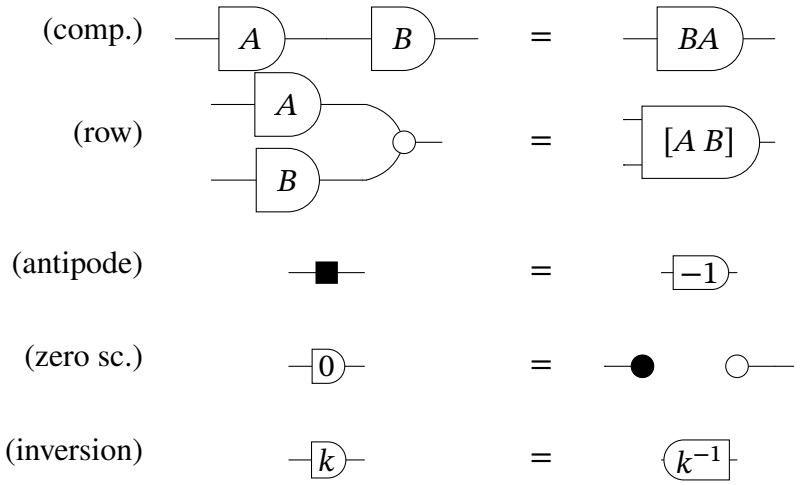
$$[\![-]\!] : \mathbf{Graphical\ Affine\ Algebra}_{\mathbb{K}} \xrightarrow{\cong} \mathbf{AffRel}_{\mathbb{K}}.$$

Consequently, Graphical Affine Algebra is sound, complete, and expressive for affine relations [Bon+19].

The theorem explains why Graphical Affine Algebra is useful in the main text. A diagram is not merely a picture of a chosen matrix calculation: it denotes the entire solution relation of an affine system. Sequential composition existentially eliminates internal variables, tensor combines independent systems, and reflection exchanges a relation with its converse. Diagram rewriting is therefore a complete algebra of affine systems.

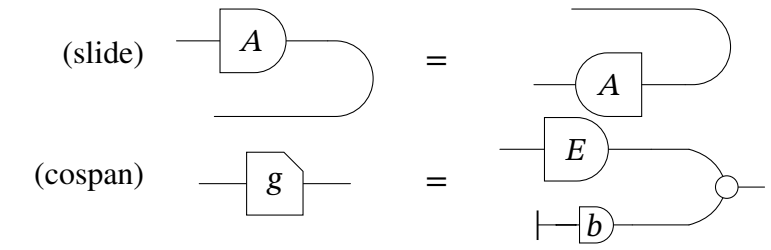
The particular derived equalities invoked by the proofs in Chapters 4 and 5 are collected below. They are grouped as (G1) matrix calculus, (G2) transposition and cospan form, (G3) bending, and (G4) the black spider laws. All follow from the Graphical Linear Algebra and Graphical Affine Algebra equations summarized above and are included here so that later diagrammatic derivations can be checked without expanding every matrix box into scalar generators.

(G1) Matrices [BSZ17; PRS22]



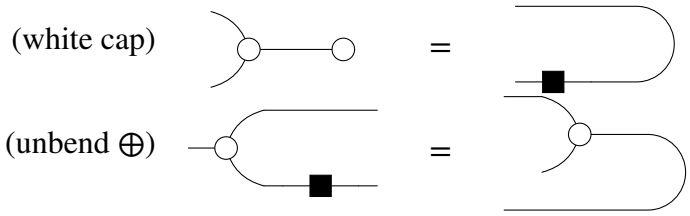
(G2) Transposition and cospan form [BSZ17]

A diagram may be slid along a cap, exchanging it for its transpose; on **GLA** diagrams the transpose is the converse relation, so a matrix becomes a comatrix.



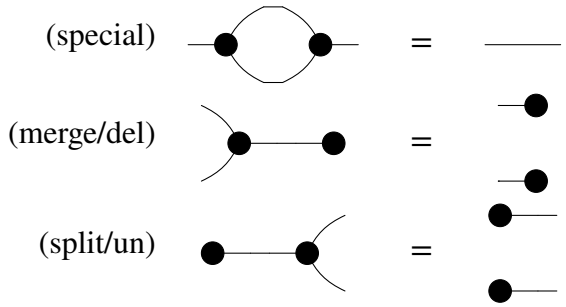
(G3) Bending [BSZ17; Bon+19]

The white cap is the black cap with an antipode on one leg; this is what turns an equation into a pair of bendable wires.



(G4) The black structure is a spider [BSZ17]

Any connected network of black nodes with a inputs and b outputs equals the single node $\mathsf{cc}_a;\mathsf{cp}_b$; the two laws below are the cases used here.



The affine point additionally turns homogeneous matrix equations into affine ones: together with copying and scalar multiplication it constructs any constant vector b, and hence any constraint $Ex + b = Fy$. The later calculi in this thesis extend this affine base with order and optimization generators. Their new equations are studied in the later chapters, whereas the affine rules on which they rely are established here. The categorical background is developed in Appendix **??**.

4 POLYHEDRAL RELATIONS

We now turn to a combinatorially richer class of objects: polyhedra and polyhedral cones. These are the fundamental geometric structures underlying linear programming, and they admit a categorical treatment entirely analogous to the one developed for linear and affine relations. As before, composition is governed by polyhedral projection, which is precisely variable elimination, and it is this operation that categorical composition will simulate diagrammatically.

The morphisms encode feasibility constraints through linear inequalities. Through the quantale framework, these constraints have indicator costs 0 and $+\infty$. Their diagrammatic axiomatization is a Symmetric Monoidal Inequality Theory (SMIT) [BDS21]. This theory provides the structural foundation for representing linear programs, where feasibility constraints are combined with a linear objective.

In what follows we present the semantics from polyhedral cones to general polyhedra, then construct the corresponding graphical algebras in the same order. The normal forms and comparison results are stated as facts of the existing theory; only soundness, expressivity, and completeness are proved here.

Recall how we moved from Graphical Linear Algebra to Graphical Affine Algebra: we added one generator that allowed us to write constants. Here we will add a generator that lets us write inequalities. If we start without the affine point, we get polyhedral cones. If we include it, we get general polyhedra. The four categories fit this simple distinction:

	without constants	with constants
equations	$\mathbf{LinRel}$	$\mathbf{AffRel}$
inequalities	$\mathbf{PCRel}$	$\mathbf{PolyRel}$

We do not lose equations when we add inequalities: $u = v$ can be written as $u \geq v$ and $v \geq u$. Nor do we change how relations compose. We simply gain more ways to describe them.

4.1 SEMANTICS: FROM POLYHEDRAL CONES TO POLYHEDRA

We begin by recalling the standard definitions of polyhedral cones and the corresponding semantic category.

Definition 4.1. *A set $C \subseteq \mathbb{R}^n$ is called a* polyhedral cone *if there exists a matrix $A \in \mathbb{R}^{m \times n}$ such that*

$$C = \{x \in \mathbb{R}^n \mid Ax \geq 0\}.$$

We use polyhedral cones as morphisms to expose their compositional and monoidal structure. A cone in a product space defines a binary relation and can equivalently be represented by its

indicator function.

Definition 4.2 (The prop $\mathbf{PCRel}$ of polyhedral cones)**.** *Let k be an ordered field, in this case the real numbers. The prop $\mathbf{PCRel}_k$ is defined as follows:*

- *Objects are natural numbers $n \in \mathbb{N}$, interpreted as the space k^n.*
- *A morphism $n \to m$ is a polyhedral cone $R \subseteq k^n \times k^m$ for which there exist $p \in \mathbb{N}$ and $A \in k^{p \times (n+m)}$ such that*

$$R = \left\{ (x, y) \in k^n \times k^m \;\middle|\; A \begin{pmatrix} x \\ \\ y \end{pmatrix} \geq 0 \right\}.$$

- *Composition is relational composition.*
- *The monoidal product on objects is addition $n \otimes m := n + m$, and on morphisms is the Cartesian product.*

A polyhedral cone relation can be described by pairs (x, y) satisfying $Ax \geq By$ for suitable matrices A and B. Composition is relational composition. Polyhedral cones are closed under this operation and under Cartesian product, so they can be combined sequentially or in parallel.

Notice the connection with the previous chapter: where we could write $Ax = By$, we can now write $Ax \geq By$. We still have no constants, and setting all variables to zero still satisfies the constraints. This is why Graphical Linear Algebra is our starting point for the language of cones. Adding the inequality generator will let us describe exactly the relations in $\mathbf{PCRel}$.

The same construction extends from polyhedral cones to general polyhedra.

Definition 4.3. *A set $P \subseteq \mathbb{R}^n$ is called a* polyhedron *if there exist $A \in \mathbb{R}^{m \times n}$ and $b \in \mathbb{R}^m$ such that*

$$P = \{x \in \mathbb{R}^n \mid Ax \geq b\}.$$

A polyhedral cone is the special case $b = 0$. General polyhedra admit affine constraints and may be empty; they need not contain the origin. In that light, one can define the category of polyhedral relations as:

Definition 4.4 (The prop $\mathbf{PolyRel}$ of polyhedra)**.** *Let k be an ordered field. The prop $\mathbf{PolyRel}_k$ is defined as follows:*

- *Objects are natural numbers $n \in \mathbb{N}$, interpreted as the space k^n.*

- *A morphism $n \to m$ is a polyhedron $R \subseteq k^n \times k^m$ for which there exist $p \in \mathbb{N}$, $A \in k^{p \times (n+m)}$, and $b \in k^p$ such that*

$$R = \left\{ (x, y) \in k^n \times k^m \;\middle|\; A \begin{pmatrix} x \\ \\ y \end{pmatrix} + b \geq 0 \right\}.$$

- *Composition is relational composition.*

- *The monoidal product on objects is addition $n \otimes m := n + m$, and on morphisms is the Cartesian product.*

As with cone relations, a polyhedral relation may be described directly by constraints on its input and output values: $xRy \iff Ax + b \geq By + c$ for suitable matrices A, B and vectors b, c. This is a relational formulation of the same inequality description.

Once again, the affine point makes the difference. Without it we write $Ax \geq By$; with it we can write $Ax + b \geq By$. We can therefore start with Graphical Linear Algebra, add inequalities, and then add the affine point. Or we can start with Graphical Affine Algebra and add inequalities. Including the axioms for both generators gives the same language of polyhedra.

This is what we mean by the affine version of cones: we allow constants in the inequalities. We do not mean that every polyhedron is a shifted cone; a bounded interval is already a counterexample. Cones remain a special case, so every relation in $\mathbf{PCRel}$ is also a relation in $\mathbf{PolyRel}$, composed in exactly the same way.

In both categories, relational composition eliminates the shared variables. Projection and Cartesian product preserve polyhedral cones and polyhedra, so these operations stay within the stated classes [BDS21, Definition 2 and Section 5]. The hom-sets are ordered by inclusion of relations.

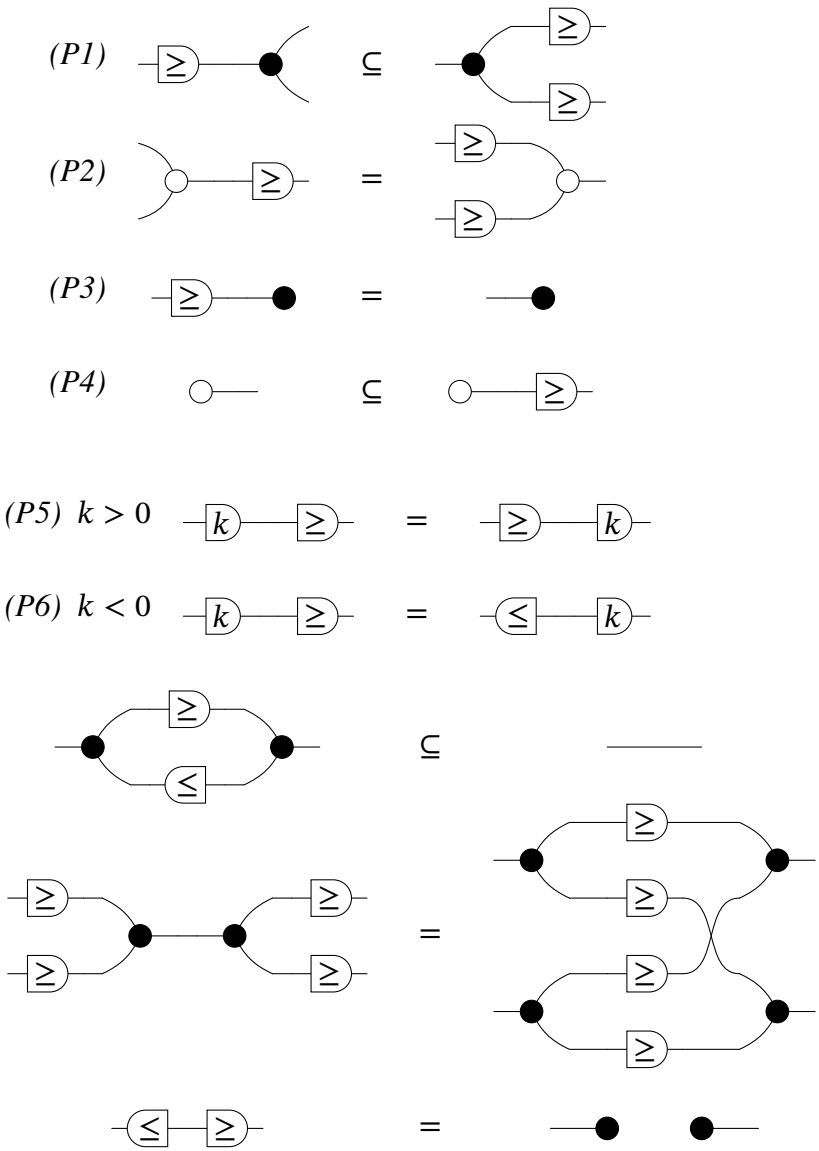
We construct the diagrammatic language in two stages. First, we extend Graphical Linear Algebra with inequalities to represent $\mathbf{PCRel}$. We then add the affine point to represent general polyhedra.

4.2 SYNTAX: FROM THE CONIC FRAGMENT TO GRAPHICAL POLYHEDRAL ALGEBRA

We start with the smaller language from Definition 3.3: Graphical Linear Algebra, without the affine point. We keep its generators and equations, and add the inequality generator with its axioms. This is the ‘‘linear fragment of Graphical Affine Algebra’’ mentioned below.

We begin with the linear fragment of the previous chapter and add the inequality generator  $: 1 \to 1$, interpreted as $[\![$ $]\!] = \{(x, y) \in k^2 \mid x \geq y\}$.

Definition 4.5 (Graphical Polyhedral Cones Algebra)**.** *We call* $\mathbf{GPA}_{\vdash}$ *the SMIT obtained by extending the linear fragment of Graphical Affine Algebra with the inequality generator. Its additional equations and inequations are the following:*



This fragment axiomatises polyhedral cones, where no affine offset is present.

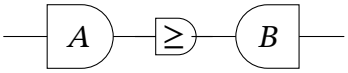
The linear fragment excludes the $\vdash$ generator and its axioms; it is precisely Graphical Linear Algebra. As in the previous chapter, we use the extended theory to generate a PROP.

Definition 4.6 (The PROP generated by the conic theory)**.** *We write* $\mathrm{FreeP}_{\mathbf{GPA}_{\vdash}}$ *for the ordered PROP generated by Definition 4.5. Its objects are natural numbers, and its terms* $m \to n$ *are diagrams built from the linear generators and* $\geq$ *by composition and tensor. The order is generated by the displayed inequations, the equations in both directions, and the symmetric monoidal structural laws, closed under composition and tensor. Morphisms are terms identified when the order is derivable in both directions.*

An inequation records inclusion of relations; equality means that the inclusions hold in both directions. Interpreting the generators gives $[\![\cdot]\!] : \mathrm{Free}\mathrm{P}_{\mathbf{GPA}_{\vdash}} \to \mathbf{PCRel}$ once the soundness checks below establish that the interpretation respects these identifications.

As in Graphical Affine Algebra, every diagram admits a normal form.

Theorem 4.1 (First Normal Form)**.** *Every diagram in* $\mathbf{GPA}_{\vdash}$ *is equal, modulo the axioms, to a diagram of the following form:*



Its interpretation is $Ax \geq By$, the standard inequality description of a polyhedral cone relation. Bending the external inputs into outputs gives a bijection on hom-sets and allows the relation to be regarded as a cone in k^{m+n}.

Proposition 4.1 (Finitely generated normal form)**.** *After bending its inputs into outputs, every diagram in* $\mathbf{GPA}_{\vdash}$ *is equal to a diagram* $d_V : 0 \to N$*, where* N *is the total number of external wires and*

$$[d_V] = \{V\lambda \mid \lambda \in k^r,\ \lambda \geq 0\}$$

for some $V \in k^{N \times r}$.

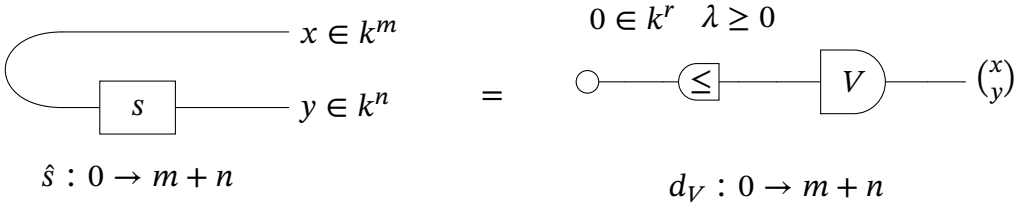


Figure 4.1 – Bending the inputs of $s : m \to n$ gives a diagram $\hat{s} : 0 \to m + n$, with the x coordinates first. Its finitely generated normal form d_V generates $\lambda \geq 0$ and applies $V : k^r \to k^{m+n}$. Each labelled wire represents a bundle of coordinates.

Here d_V is the diagram that generates r non-negative values and applies the linear map V. This is the second normal form: the relation is described by non-negative combinations of the columns of V. Neither normal form is unique. The existence of these forms is established in [BDS21, Theorems 14 and 18]; the technical derivations are collected in Appendix **??**.

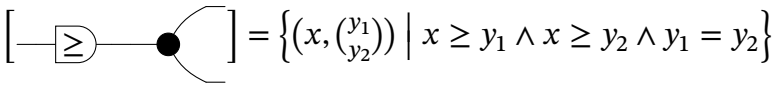
Lemma 4.1 (Comparison of finitely generated diagrams)**.** *Let* $V \in k^{N \times r}$ *and* $W \in k^{N \times q}$*. If* $V = WC$ *for a matrix* $C \in k^{q \times r}$ *with non-negative entries, then* $d_V \subseteq d_W$ *is derivable in* $\mathbf{GPA}_{\vdash}$.

This is the comparison fact needed for completeness. It states that a non-negative combination of generating columns gives a diagrammatic inclusion [BDS21, Lemmas 49–50].

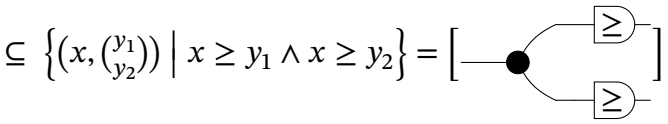
Lemma 4.2 (Soundness)**.** *If* $s = t$ *in* $\mathrm{Free}\mathrm{P}_{\mathbf{GPA}_{\vdash}}$*, then* $[s] = [t]$ *in* $\mathbf{PCRel}$.

Proof. All axioms except those involving the newly introduced $\geq$ generator are already present in $\mathbf{GAA}$ and therefore verified. It suffices to check that the new axioms preserve the semantics:

(P1)

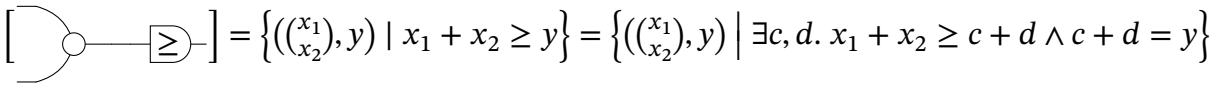


$$= \left\{ \left(x, \binom{y_1}{y_2}\right) \;\middle|\; x \geq y_1 \wedge x \geq y_2 \wedge y_1 = y_2 \right\}$$

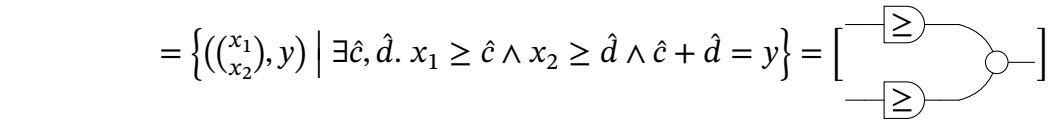


$$\subseteq \left\{ \left(x, \binom{y_1}{y_2}\right) \;\middle|\; x \geq y_1 \wedge x \geq y_2 \right\} =$$

(P2)

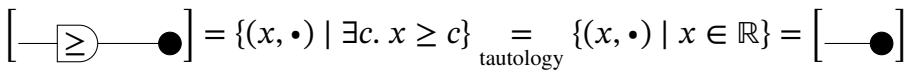


$$= \left\{ \left(\binom{x_1}{x_2}, y\right) \mid x_1 + x_2 \geq y \right\} = \left\{ \left(\binom{x_1}{x_2}, y\right) \;\middle|\; \exists c, d.\; x_1 + x_2 \geq c + d \wedge c + d = y \right\}$$



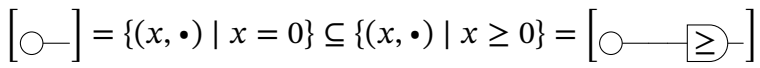
$$= \left\{ \left(\binom{x_1}{x_2}, y\right) \;\middle|\; \exists \hat{c}, \hat{d}.\; x_1 \geq \hat{c} \wedge x_2 \geq \hat{d} \wedge \hat{c} + \hat{d} = y \right\} =$$

(P3)



$$= \{(x, \bullet) \mid \exists c.\; x \geq c\} \underset{\text{tautology}}{=} \{(x, \bullet) \mid x \in \mathbb{R}\} =$$

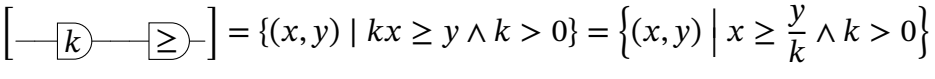
(P4)



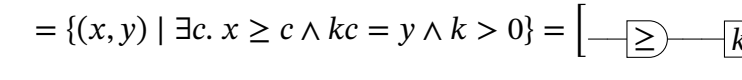
$$= \{(x, \bullet) \mid x = 0\} \subseteq \{(x, \bullet) \mid x \geq 0\} =$$

(P5), (P6)

For $k > 0$:

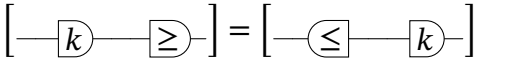


$$= \{(x, y) \mid kx \geq y \wedge k > 0\} = \left\{ (x, y) \;\middle|\; x \geq \frac{y}{k} \wedge k > 0 \right\}$$

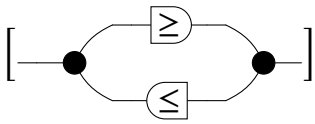


$$= \{(x, y) \mid \exists c.\; x \geq c \wedge kc = y \wedge k > 0\} =$$

For $k < 0$: the same computation, dividing by the negative k flips the inequality, so



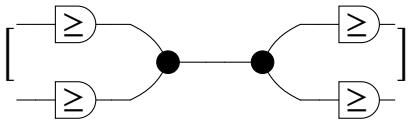
(ANTISYM)



$$= \{(x,y) \mid x \geq y \wedge x \leq y\}$$

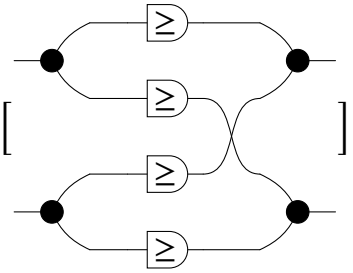
$$= \{(x,y) \mid x = y,\ x,y \in \mathbb{R}\} \subseteq \{(x,y) \mid x = y,\ x,y \in \mathbb{R}\} =$$

(SPIDER)



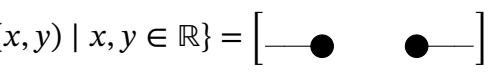
$$= \left\{ \left(\binom{x_1}{x_2}, \binom{y_1}{y_2} \right) \;\middle|\; \exists z.\ x_1 \geq z \wedge x_2 \geq z \wedge z \geq y_1 \wedge z \geq y_2 \right\}$$

$$= \left\{ \left(\binom{x_1}{x_2}, \binom{y_1}{y_2} \right) \;\middle|\; x_1 \geq y_1 \wedge x_1 \geq y_2 \wedge x_2 \geq y_1 \wedge x_2 \geq y_2 \right\} =$$



(DIRECTION)

$$= \{(x,y) \mid \exists z.\ z \geq y \wedge z \geq x\} = \{(x,y) \mid x,y \in \mathbb{R}\} =$$

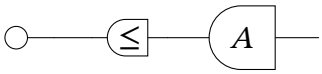


□

The same checks show that the inequations are interpreted as inclusions.

Lemma 4.3 (Expressivity)**.** *For every $f : m \to n$ in* $\mathbf{PCRel}$ *there exists a term $s : m \to n$ in* $\mathrm{Free}\mathrm{P}_{\mathbf{GPA}_{\vdash}}$ *such that* $[\![s]\!] = f$.

Proof. By Definition 4.1, every polyhedral cone has the form $\{x \mid Ax \geq 0,\ x \in \mathbb{R}^n\}$. This is precisely the interpretation of the diagram:



□

Here the diagram expresses a cone with all its coordinates as outputs. Bending the first m wires back into inputs gives the required type $m \to n$.

Lemma 4.4 (Completeness)**.** *If* $[s] = [t]$ *in* $\mathbf{PCRel}$*, then* $s = t$ *in* $\mathrm{Free}\mathrm{P}_{\mathbf{GPA}_{\vdash}}$.

Proof. Bend the inputs into outputs and use the finitely generated normal form to write the two diagrams as d_{V_s} and d_{V_t}. Their semantics are the conic hulls of the columns of V_s and V_t. Since $[s] = [t]$, each column of V_s is a non-negative combination of columns of V_t. Collecting the coefficients gives $V_s = V_t C$ for a non-negative matrix C. Lemma 4.1 yields $d_{V_s} \subseteq d_{V_t}$. Reversing the roles of s and t yields the opposite inclusion, hence equality. Unbending the wires recovers $s = t$. □

Theorem 4.2. $\mathbf{PCRel}$ *is presented by* $\mathrm{Free}\mathrm{P}_{\mathbf{GPA}_{\vdash}}$.

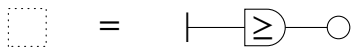
Together, soundness, expressivity, and completeness establish this presentation [BDS21, Corollary 21].

Cones always contain the origin, so they cannot represent an arbitrary feasible region. We now reintroduce the affine generator $\vdash$ to encode the right-hand side vector b in $Ax \geq b$ and extend the theory to general polyhedra.

We already have a language for cones, what is missing are the constants. We add the affine point with its three equations from Definition 3.4, and (P0), which tells us how it interacts with the inequality generator. This gives the language of polyhedra. We could also have started with Graphical Affine Algebra and added the inequality generator with its axioms and (P0); either way, we have the same generators and rules [BDS21, Section 5].

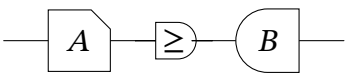
Definition 4.7 (Graphical Polyhedral Algebra)**.** *We call* $\mathbf{GPA}$ *the SMIT obtained by adjoining the affine generator* $\vdash : 0 \to 1$*, interpreted as the constant* 1*, to* $\mathbf{GPA}_{\vdash}$*. We include the affine equations of Affine Algebra and the following additional axiom:*

(P0)



The inequality generator and its homogeneous axioms remain those of Definition 4.5; only (P0) is new. We write $\mathrm{Free}\mathrm{P}_{\mathbf{GPA}}$ for the ordered PROP generated by this extended theory, using the same construction as in Definition 4.6.

Theorem 4.3 (Normal Form)**.** *For every diagram* c *in* $\mathbf{GPA}$*, there exists a diagram* d *such that* $c = d$ *modulo the axioms and* d *is of the form:*



The affine block records the offsets, so the interpretation has the form $Ax + b \geq By$. The normal form is established in [BDS21, Theorem 24]; its derivation is collected in Appendix **??**.

For completeness, we use the following two facts of the existing theory. The first concerns non-empty relations; the second handles infeasibility.

Lemma 4.5 (Homogeneous comparison)**.** *For non-empty polyhedra* $P_i = \{z \in k^N \mid A_i z + b_i \geq 0\}$*, define*

$$P_i^H = \{(z, \lambda) \mid A_i z + b_i \lambda \geq 0,\ \lambda \geq 0\}.$$

Then $P_1 \subseteq P_2$ *if and only if* $P_1^H \subseteq P_2^H$*. Replacing the affine constant in a normal-form diagram by a shared non-negative variable gives its homogeneous diagram. Derivable comparisons of those homogeneous diagrams give comparisons of the original diagrams by setting that variable to* 1*.*

These facts are established in [BDS21, Lemma 25 and Theorem 26]. Their role is to let us use the complete conic calculus for non-empty polyhedral relations.

Lemma 4.6 (Empty normal form)**.** *For each type* $m \to n$*, all diagrams of* $\mathbf{GPA}$ *whose interpretation is empty are equal to the same empty diagram* $\bot_{m,n}$*.*

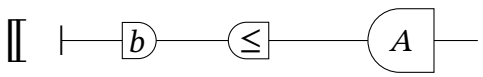
This fact is established in [BDS21, Theorem 27]. It is essential: the homogeneous comparison above assumes non-empty polyhedra and cannot by itself cover infeasible constraints.

Lemma 4.7 (Soundness)**.** *If* $s = t$ *in* $\mathrm{FreeP}_{\mathbf{GPA}}$*, then* $[s] = [t]$ *in* $\mathbf{PolyRel}$*.*

Proof. The homogeneous axioms are sound by the conic case, and the affine axioms inherited from $\mathbf{GAA}$ are already sound. The remaining axiom (P0) is valid because the constant 1 satisfies $1 \geq 0$, so its scalar diagram has the same interpretation as the empty diagram. Thus all axioms preserve the polyhedral semantics. □

Lemma 4.8 (Expressivity)**.** *For every* $f : m \to n$ *in* $\mathbf{PolyRel}$ *there exists a term* $s : m \to n$ *in* $\mathrm{FreeP}_{\mathbf{GPA}}$ *such that* $[s] = f$*.*

Proof. By Definition 4.3, every polyhedron has the form $\{x \mid Ax \geq b\}$. Appealing to this representation directly, the polyhedron is expressed as the interpretation of the following diagram, where the generator $\vdash$ encodes the right-hand side vector b:



$$= \{x \in \mathbb{R}^N \mid Ax \geq b\}$$

Indeed, the interpretation is a symmetric monoidal functor, so the diagram denotes the relational composite of the semantics of its three components:

$$[\![\vdash; b]\!] = \{(\bullet, b)\}, \qquad [\![\text{—}\leq\text{—}]\!] = \{(u, w) \mid w \geq u\}, \qquad [\![A^{\mathrm{op}}]\!] = \{(w, x) \mid w = Ax\}.$$

Composing them eliminates the internal wires:

$$\{(\bullet, x) \mid \exists\, w :\ w \geq b,\ w = Ax\} = \{x \in \mathbb{R}^N \mid Ax \geq b\},$$

which is the claimed polyhedron. With $b = 0$, this is the diagram of the conic case. □

As in the conic case, bending the first m coordinates into inputs gives a diagram with the required interface.

Lemma 4.9 (Completeness)**.** *If* $[s] = [t]$ *in* $\mathbf{PolyRel}$*, then* $s = t$ *in* $\mathrm{FreeP}_{\mathbf{GPA}}$.

Proof. If the common interpretation is empty, Lemma 4.6 gives $s = \bot_{m,n} = t$.

Otherwise, put both diagrams in affine normal form. Lemma 4.5 gives equal homogeneous interpretations. Completeness of the conic fragment gives equality of the homogeneous diagrams. Setting the shared homogeneous variable to 1 recovers equality of the affine normal forms, hence $s = t$. □

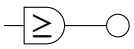
Theorem 4.4. $\mathbf{PolyRel}$ *is presented by* $\mathrm{FreeP}_{\mathbf{GPA}}$.

This follows from soundness, expressivity, and completeness [BDS21, Corollary 30]. Every polyhedral relation has a diagrammatic representative, and semantic equality can be certified in $\mathbf{GPA}$. It is this presentation, together with its normal form, that Chapter 5 builds on when it adds the generator for the objective function.

4.3 THE TRANSPORTATION CONSTRAINTS REVISITED

Recall the divisible-traffic example from Chapter 1. We now draw its constraints in Graphical Polyhedral Algebra, without the monetary objective. Every node in the diagram below is a generator; nothing is abbreviated. We follow the two-stage plan of Chapter 1: the fuel budget is split as $F = f + g$, with f allocated to the first stage and g to the second, and the stages share the amount q sent through B.

One small pattern occurs three times, so we name it first. The diagram



denotes $\{(x, *) \mid x \geq 0\}$: the inequality generator gives $x \geq y$ and the white counit forces $y = 0$. We call it the *non-negativity test*.

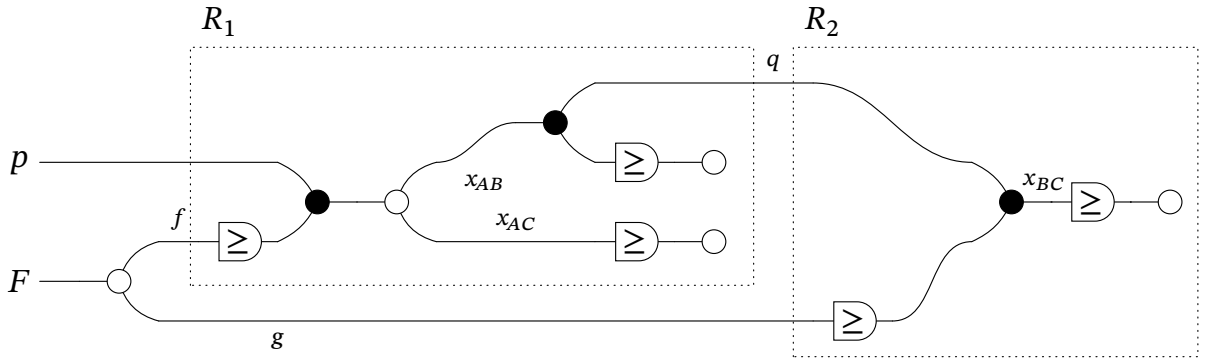


Figure 4.2 – The transportation constraints in Graphical Polyhedral Algebra. The dotted boxes group the two stages $R_1 : 2 \to 1$ and $R_2 : 2 \to 0$; they are not new generators. The wires f, g, and q are internal.

We read the diagram from left to right.

- *Fuel split.* The white comultiplication on F imposes $F = f + g$.
- *First stage* R_1. The allowance f passes through  , which produces a value not exceeding f. The black node identifies this value with p, so $p \le f$. The white comultiplication then splits p into the two roads leaving A, imposing $x_{AB} + x_{AC} = p$. Since this sum is the fuel used by the first stage, the black node states that this fuel fits in f. The black copy sends x_{AB} both to the interface, as q, and to a non-negativity test; x_{AC} goes to its own test.
- *Second stage* R_2. The same pattern appears again. The allowance g passes through  and the black node identifies the result with q. The common value is the flow x_{BC} on the road $B \to C$, so $x_{BC} = q \le g$. A final non-negativity test gives $x_{BC} \ge 0$.

Conservation at B requires no node: the flow arriving at B is the wire q, and the flow leaving B is the same wire.

Composing the generators inside each box gives the two stage relations

$$(p, f)\, R_1\, q \iff \exists\, x_{AB}, x_{AC} \ge 0 :\ x_{AB} + x_{AC} = p,\ x_{AB} = q,\ p \le f,$$
$$(q, g)\, R_2\, * \iff \exists\, x_{BC} \ge 0 :\ x_{BC} = q,\ x_{BC} \le g.$$

These are the constraints of U and W in Chapter 1, with $g = F - f$. Relational composition then eliminates the internal wires f, g, and q:

$$(p, F)\,\text{accepted} \quad \iff \quad \exists\, x_{AB}, x_{BC}, x_{AC} \ge 0 :\ x_{AB}{+}x_{AC} = p,\ x_{BC} = x_{AB},\ x_{AB}{+}x_{BC}{+}x_{AC} \le F.$$

Indeed, a split $F = f + g$ with $p \le f$ and $x_{BC} \le g$ exists exactly when $p + x_{BC} \le F$, and $p + x_{BC} = x_{AB} + x_{BC} + x_{AC}$. These are precisely the original constraints. Every feasible pair satisfies $p = x_{AB} + x_{AC} \ge 0$ and $F \ge p + x_{BC} \ge p$; conversely, when $0 \le p \le F$, sending everything along the direct road is feasible. The feasible pairs are therefore $p \ge 0$ and $F \ge p$.

The diagram uses no affine point: every constraint is homogeneous, so it lives in the conic fragment $\mathbf{GPA}_{\vdash}$, and its semantics is the polyhedral cone $\{(p, F) \mid 0 \le p \le F\}$. No cost is attached at this stage: the diagram records feasibility only.

5 THE LINEAR PROGRAMMING CATEGORY

We finally reach the point of interest: the final and most expressive theory of the thesis, a graphical algebra for parametric linear programs. Linear programs combine both the inequality structure of polyhedra and the notion of an objective function. The central result of this section is the presentation of a complete monoidal inequational theory for right-hand side parametric linear programming.

5.1 SEMANTICS: THE PROP OF LINEAR PROGRAMS

The semantic category of this chapter is called $\mathbf{LPRel}$. Its morphisms are the value functions of right-hand side parametric linear programs, regarded as cost-valued relations in $\mathbf{Rel}(\overline{\mathbb{R}})$. Here $\overline{\mathbb{R}}$ carries the optimization quantale introduced in the chapter on composition and order, with inf-addition $(+\infty) + (-\infty) = +\infty$. The main objects are, therefore, LP value functions.

Definition 5.1 (LP value function)**.** *Let $m, n \in \mathbb{N}$. A function $v : \mathbb{R}^m \times \mathbb{R}^n \to \overline{\mathbb{R}}$ is an* LP value function of type $m \to n$ *if there exist $k \in \mathbb{N}$, $A \in \mathbb{R}^{p\times k}$, $B \in \mathbb{R}^{p\times m}$, $C \in \mathbb{R}^{p\times n}$, $e \in \mathbb{R}^p$ and $c \in \mathbb{R}^k$ such that*

$$v(x, y) = \inf_{z\in\mathbb{R}^k} \{c^\top z \mid Az \geq Bx + Cy + e\}, \qquad \inf \varnothing = +\infty.$$

We call (A, B, C, e, c) a presentation *of v. Different presentations may define the same value function.*

By the equivalence proved in Chapter 1, the LP value functions of type $m \to n$ are exactly the extended piecewise affine LP functions on $\mathbb{R}^{m+n}$ and as the composition of two LP value functions is the $\inf$-composition of weighted relations. Thus, we can formally define our prop.

Definition 5.2 (The prop $\mathbf{LPRel}$)**.** *The prop $\mathbf{LPRel}$ has the natural numbers as objects, with n interpreted as $\mathbb{R}^n$. A morphism $m \to n$ is an LP value function of type $m \to n$; two programs with the same value function are the* same *morphism. Composition, identities, tensor and symmetries are those of $\mathbf{Rel}(\overline{\mathbb{R}})$:*

$$(w \circ v)(x, u) = \inf_{y\in\mathbb{R}^n} \big(v(x, y) + w(y, u)\big),$$
$$(v \otimes v')\big((x, x'), (y, y')\big) = v(x, y) + v'(x', y'),$$
$$\mathrm{id}_n(x, y) = \begin{cases} 0 & x = y, \\ +\infty & \textit{otherwise}, \end{cases}$$

where the sums use inf-addition. The symmetry is defined like the identity, with the coordinates swapped. Hom-sets carry the quantale order

$$v \sqsubseteq w \iff v(x, y) \geq w(x, y) \textit{ for all } (x, y),$$

that is, a morphism is smaller when it is more costly.

For $\mathbf{LPRel}$ to be a sub-prop of $\mathbf{Rel}(\overline{\mathbb{R}})$, the LP value functions must be closed under these operations. This is the point where the inf-addition convention matters. It is easy to check that the composition of LPs is indeed another LP; we state the formal lemma and give a brief proof.

Lemma 5.1 (Closure)**.** *Identities and symmetries are LP value functions, and LP value functions are closed under composition and tensor. Composition and tensor are monotone with respect to* $\sqsubseteq$*. Hence* $\mathbf{LPRel}$ *is an ordered sub-prop of* $\mathbf{Rel}(\overline{\mathbb{R}})$*.*

Proof. Identities and symmetries are presented with no decision variables, by pairs of opposite inequalities. For composition, put the two programs side by side, add their costs, and treat the shared interface y as an extra decision variable with zero cost. The result is again an RHS-parametric program in the outer variables. Tensor is the same construction without a shared variable. The only point to check is the boundary case: if one stage is infeasible and the other unbounded, the merged program is infeasible. This matches inf-addition, $(+\infty) + (-\infty) = +\infty$. Monotonicity holds because infima and inf-addition are monotone in each argument. □

Polyhedral relations sit inside $\mathbf{LPRel}$ as the programs whose cost is identically zero. As in the chapter on composition and order, we identify a relation P with the cost-valued relation that is 0 on pairs in P and $+\infty$ elsewhere, and we use the same letter for both.

Proposition 5.1 (Polyhedral relations in $\mathbf{LPRel}$)**.** *This identification defines an identity-on-objects strict symmetric monoidal functor* $\mathbf{PolyRel} \to \mathbf{LPRel}$*. The functor is faithful, and it is an order embedding:* $P \subseteq Q$ *if and only if* $P \sqsubseteq Q$*.*

Proof. If $P = \{(x, y) \mid Bx + Cy + e \leq 0\}$, then P, read as a cost-valued relation, is presented with $k = 0$ by the constraints $0 \geq Bx + Cy + e$. It therefore lies in $\mathbf{LPRel}$. Composing in $\mathbf{LPRel}$,

$$\inf_y \big(P(x, y) + Q(y, u)\big)$$

is 0 exactly when some y satisfies $(x, y) \in P$ and $(y, u) \in Q$, and $+\infty$ otherwise. This is the relational composite $Q \circ P$. Tensor, identities and symmetries agree in the same way, because $0 + 0 = 0$ and $+\infty$ is absorbing. Finally, $P(x, y) \geq Q(x, y)$ holds for all (x, y) exactly when every pair in P lies in Q. This gives faithfulness and the order embedding. □

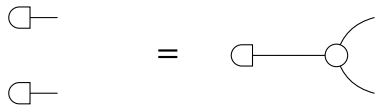
Observe how composition in $\mathbf{LPRel}$ recalls the composition of value functions derived in Chapter 1. We now build it into the syntax, so that diagrams handle the bookkeeping of the infima for us and we can work with the components directly.

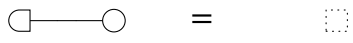
The algebra **Graphical Linear Programming** extends **Graphical Polyhedral Algebra** with a single new generator whose semantics encodes the linear objective function $c^\top x$. This new generator lifts **Graphical Polyhedral Algebra** from polyhedral constraints to full linear programs by attaching a linear cost to the feasible region.

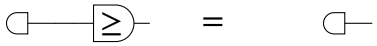
Definition 5.3 (**Graphical Linear Programming**). *We call* ***Graphical Linear Programming*** *the symmetric monoidal theory obtained by extending* ***Graphical Polyhedral Algebra*** *with the following generator:*

$$[\![\text{⊂—}]\!] = (\bullet, x) \mapsto x$$

together with the following axioms governing its interaction with the existing generators:

(linearity) 

(zeroeq) 

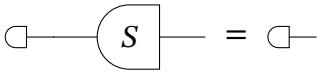
(inf) 

The axioms represent expected properties of the linear cost function.

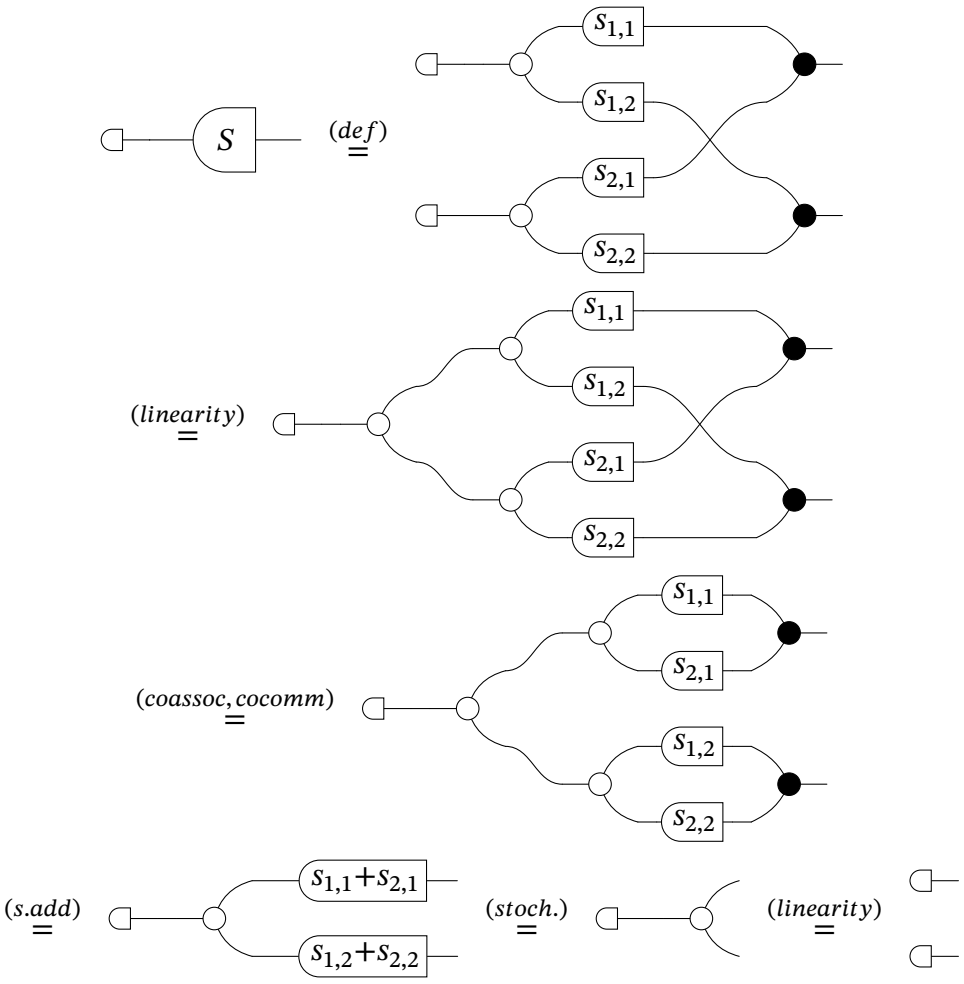
- (linearity) express the linear property $f(x) + f(y) = f(x + y)$. Observe that f is the interpretation of the ⊂— and the sum denotes the vertical composition; composing two ⊂— vertically (adding) is the same as adding the wires in a single ⊂—.
- (zeroeq) is the tautological fact that $\inf 0 = 0$
- (inf) is the epigraph property, this is, the fact that the infimum of the epigraph of a function, is the function itself $g(y) = \inf\{x : (y, x) \in \mathrm{epi}(f)\} = \inf_{x \geq f(y)} x = f(y)$.

From these simple axioms, we immediately get another core property of linear programs, namely, that the cost generator is invariant under matrices whose columns sum to one.

Remark 5.1 (Invariance under matrices with unit column sums)**.** *The linearity axiom encodes a fundamental invariance of linear objectives under reweightings that preserve the total: for any $n \times m$ matrix S whose columns each sum to one,*



The proof is diagrammatic, we display the proof for a 2×2 matrix S with $s_{1,1}+s_{2,1} = s_{1,2}+s_{2,2} = 1$; every step is an instance of an axiom scheme that applies wire-by-wire, so the same chain of rewrites goes through verbatim for arbitrary $n \times m$.



No sign condition is required on the entries of S. When they are also non-negative, S is column-stochastic, which gives the remark its familiar reading.

This is a pattern present in every algebra discussed so far, where a particular class of matrices is absorbed by one of its core generators. [1] Here, the class of matrices that interacts with the new generator ⊂— is that of matrices whose columns sum to one, agreeing with what one would expect when discussing linear problems.

Graphical Polyhedral Algebra is a theory with inequations, and Graphical Linear Programming inherits them. Its free prop is therefore ordered, in the same way as in Definition 4.6.

Definition 5.4. *We write* $\mathrm{Free}\mathrm{P}_{\mathbf{GLP}}$ *for the ordered prop generated by Graphical Linear Programming. Its terms* $m \to n$ *are built from the generators of* **GPA** *and* ⊂— *by composition and tensor. Its order is the smallest precongruence containing the inequations of* **GPA**, *both*

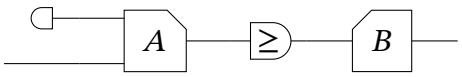
[1] In the diagrammatic calculus of linear subspaces (Graphical Linear Algebra), a ●— followed by a surjective matrix is again a ●—, which is an important feature in the completeness of the language itself. The same occurs in Graphical Polyhedral Algebra, where non-negative matrices slide through the —(≥)—. Other works have spotted the same pattern, such as [Ste+24], which shows that a generator representing the ℓ_2 norm is invariant under orthogonal matrices, mirroring the rotation invariance of the standard normal distribution.

directions of every equation (old and new), and the symmetric monoidal structural laws. Two terms are equal when the order holds in both directions.

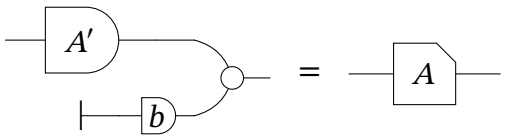
To prove completeness of this language we will appeal to the usual recipe of defining a normal form and using it as a cornerstone through which we will factor our objects. That is, we show that every object can be reduced to this normal form and, if the semantics of two normal forms are the same, that it is possible to equationally turn one into the other.

The normal form theorem for Graphical Linear Programming extends the one for Graphical Polyhedral Algebra: every diagram can be reduced to a **Graphical Polyhedral Algebra** block encoding the constraints, composed with the new generator encoding the objective. The proof proceeds by structural induction, using the existing normal form for $\mathbf{GraphicalPolyhedralAlgebra}$ as the inductive base.

Theorem 5.1 (Normal Form)**.** *For every diagram $c \in \mathbf{GraphicalLinearProgramming}$, there exists a diagram c' in normal form such that $c = c'$:*

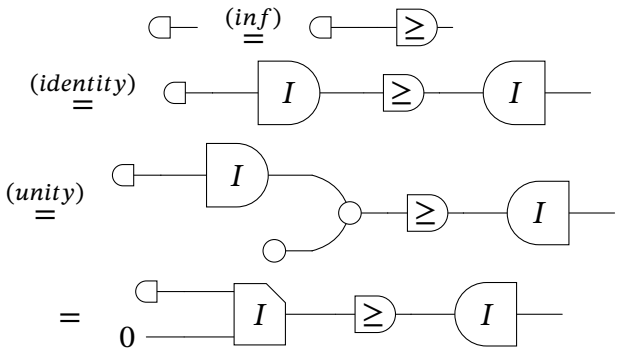


With A and b being an affine transformation. Namely:

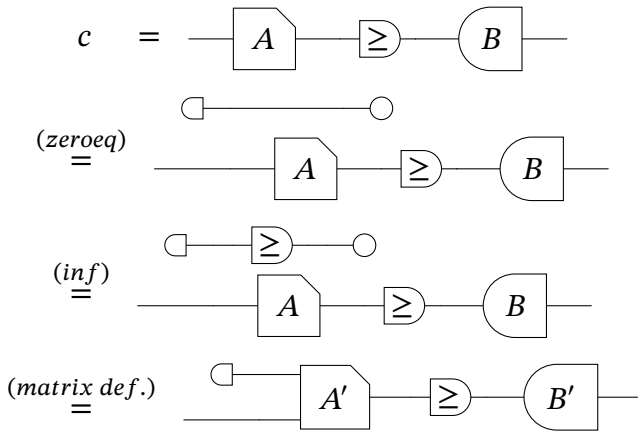


Proof. The proof is by structural induction. First we show the two base cases:

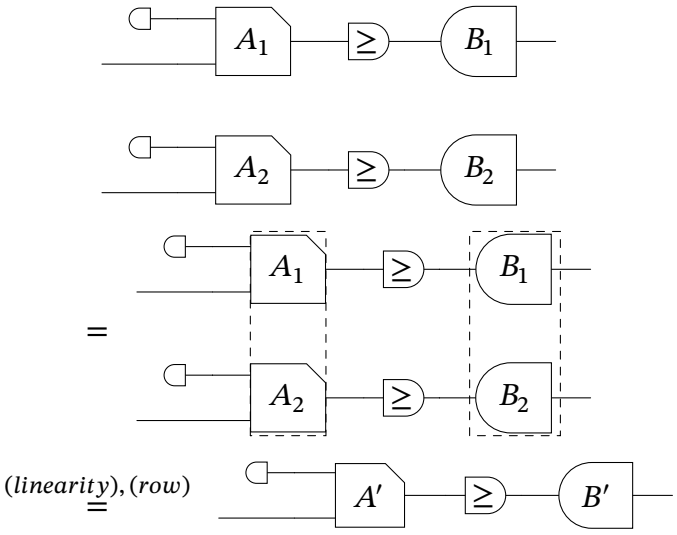
- The generator



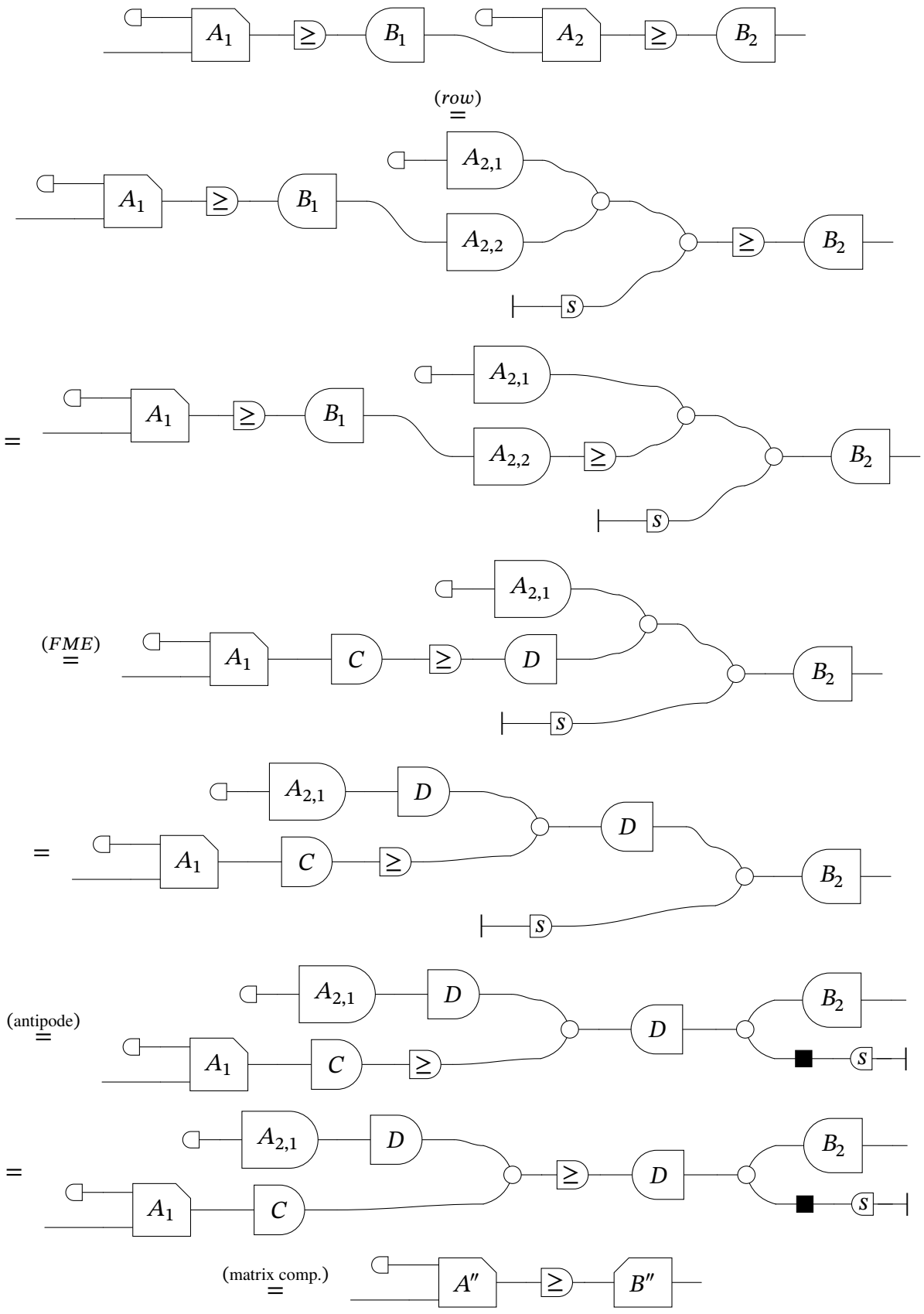
- A diagram $c \in$ **Graphical Polyhedral Algebra**



For the inductive step, we verify that for any two diagrams $c, d \in$ **Graphical Linear Programming** satisfying the inductive hypothesis, both horizontal and vertical composition preserve the normal form. Closure under vertical composition follows by stacking the two normal forms and collapsing the two independent A_i, ge, B_i blocks into a single block-diagonal matrix pair, as shown below:



For horizontal composition, the two normal forms are joined along the wire connecting B_1 to A_2; applying Fourier-Motzkin Elimination to this pair recovers a fresh matrix-ge-comatrix triple, after which the two resulting runs of matrices, one in each direction, compose into a single matrix and a single comatrix, recovering the normal form:



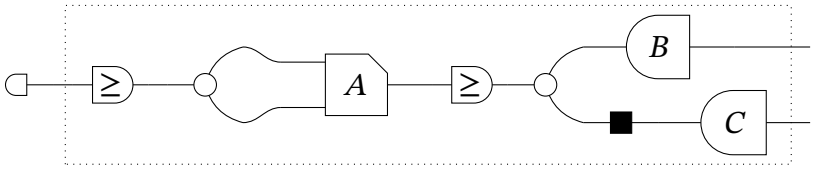
□

One can check that the semantics of this normal form is precisely the common form of

a RHS-parametric linear program. Such a normal form can be further explored to make the polyhedral structure more evident, that is, we turn the above into the epigraph form of the LP. The value function then becomes the minimization over its own epigraph, which, in turn, encodes its constraints.

The following corollary reveals the geometric meaning of the normal form: the Polyhedral part of every **GraphicalLinearProgramming** diagram encodes the epigraph of the corresponding value function. This is the key structural insight connecting the syntax to the semantic equivalence of Proposition 1.2.

Corollary 5.1 (Epigraph Form)**.** *Given a diagram $d \in$ **Graphical Linear Programming** in normal form, it can always be rewritten as a diagram d' of the form:*



Here $A = [A'\ C]$ splits the constraint block of the normal form into its action on the h cost wires z and on the m inputs y_2; the n outputs are y_1, and we write $y = (y_1, y_2)$. The semantics of the dotted diagram is

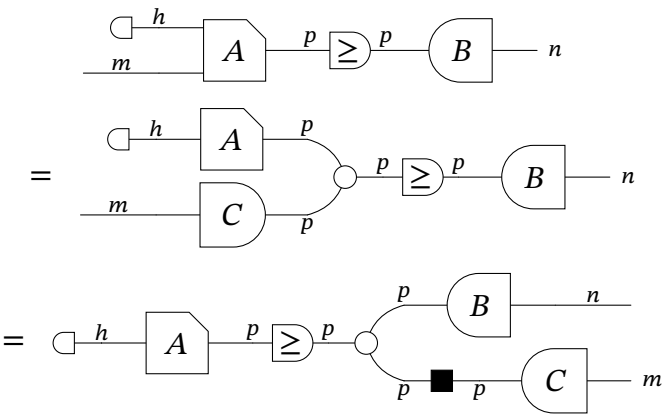
$$\left\{(t, y) \mid \exists\, z \in \mathbb{R}^h \,:\, A'z + a \geq [B\ -C]\, y,\ t \geq \mathbf{1}^\top z\right\}.$$

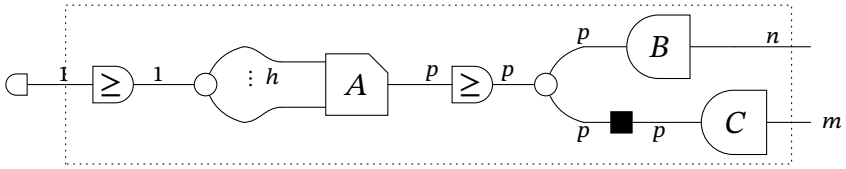
This is the epigraph of the original diagram d:

$$[d](y_1, y_2) = \inf_z\ \mathbf{1}^\top z \quad \textit{s.t. } A'z + a \geq By_1 - Cy_2,$$
$$\mathrm{epi}([d]) = \{(t, y_1, y_2) \mid t \geq [d](y_1, y_2)\}.$$

We call this the epigraph form *of a linear program.*

Proof. Starting from d in normal form, split the relation generator along $A = [A'\ C]$ and pull the resulting C-block through the ≥ generator using the antipode axiom. In the last step, (linearity) merges the h cost generators into one, and (inf) places a ≥ right after it:





Wire labels give bundle sizes: h for the cost wires, m for the inputs, n for the outputs, and p for the constraint rows.

The resulting diagram is precisely the claimed form d', with the dotted box a **Graphical Polyhedral Algebra** diagram.

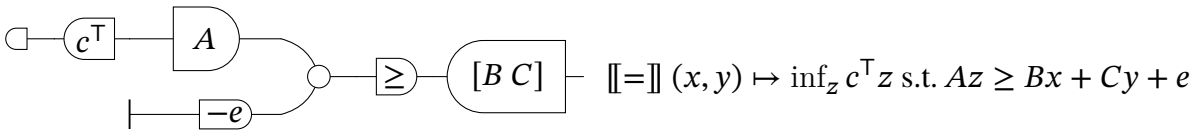
It remains to check that the dotted box denotes exactly $\mathrm{epi}([d])$. Fix y and let $S = \{\mathbf{1}^\top z \mid A'z + a \geq By_1 - Cy_2\}$, so that $[d](y) = \inf S$ and the dotted box contains (t, y) precisely when some $s \in S$ satisfies $s \leq t$. Any such t satisfies $t \geq \inf S$. Conversely, let $t \in \mathbb{R}$ with $t \geq \inf S$. If $\inf S = -\infty$, then S is unbounded below and contains some $s \leq t$. If $\inf S$ is finite, then S is the image of a nonempty polyhedron under a linear map, hence a closed interval of $\mathbb{R}$, and it contains its own infimum: this is the familiar fact that a bounded linear program attains its optimum. If $\inf S = +\infty$, there is no real t on either side. Thus the dotted box is the epigraph, including in the infeasible and unbounded cases. □

5.2 PRESENTATION RESULTS FOR $\mathbf{LPREL}$

With the syntax and its normal form in place, we are now ready to prove the three presentation properties for **Graphical Linear Programming**.

Lemma 5.2 (Expressivity)**.** *For every $f : m \to n$ in* $\mathbf{LPRel}$ *there exists a Σ-term $s : m \to n$ in* $\mathrm{Hom}_{\mathrm{FreeP}_{\mathbf{GLP}}}(m, n)$ *such that* $[s] = f$.

Proof. By Definition 5.1, every morphism of $\mathbf{LPRel}$ is the value function of some program $(x, y) \mapsto \inf_z\{c^\top z \mid Az \geq Bx + Cy + e\}$, therefore it is enough to show that we can always depict such a problem. The following diagram provides the required syntactic representative, with all interface wires drawn as outputs:

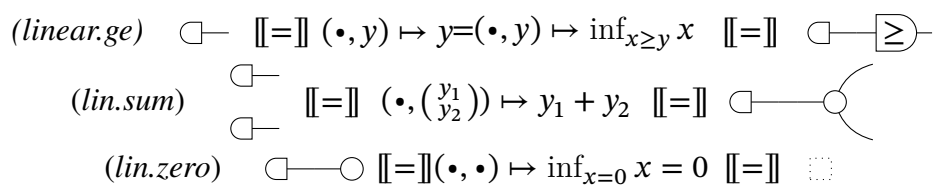


The linear generator charges the value $c^\top z$, the affine point supplies the offset e, and the reflected matrix $[B\ C]$ connects the constraints to the interface. As in the polyhedral case, bending the first m interface wires into inputs gives a diagram of type $m \to n$. □

Thus, $\mathrm{FreeP}_{\mathbf{GLP}}$ can denote any RHS-parametric LP. We move on to showing that the theory is sound, that is, that it is coherent with the semantics: diagrams identified by the axioms are never mapped to distinct LPs. In other words, the interpretation map is well defined on equivalence classes of diagrams. (Injectivity on hom-sets is the content of completeness, proved afterwards.)

Lemma 5.3 (Soundness)**.** *If $s \leq t$ in* $\mathrm{Free}\mathrm{P}_{\mathbf{GLP}}$*, then $[s] \sqsubseteq [t]$ in* $\mathbf{LPRel}$*. In particular, if $s = t$, then $[s] = [t]$.*

Proof. Since the interpretation preserves composition and tensor, and both are monotone in $\mathbf{LPRel}$ (Lemma 5.1), it suffices to check the axioms. The axioms inherited from **Graphical Polyhedral Algeb** are sound in $\mathbf{PolyRel}$ by the results of the previous chapter, and Proposition 5.1 carries them into $\mathbf{LPRel}$: an inclusion $P \subseteq Q$ becomes $P \sqsubseteq Q$, and an equality stays an equality. It remains to verify that the new axioms involving the LP generator preserve the semantics, which follows by direct computation:



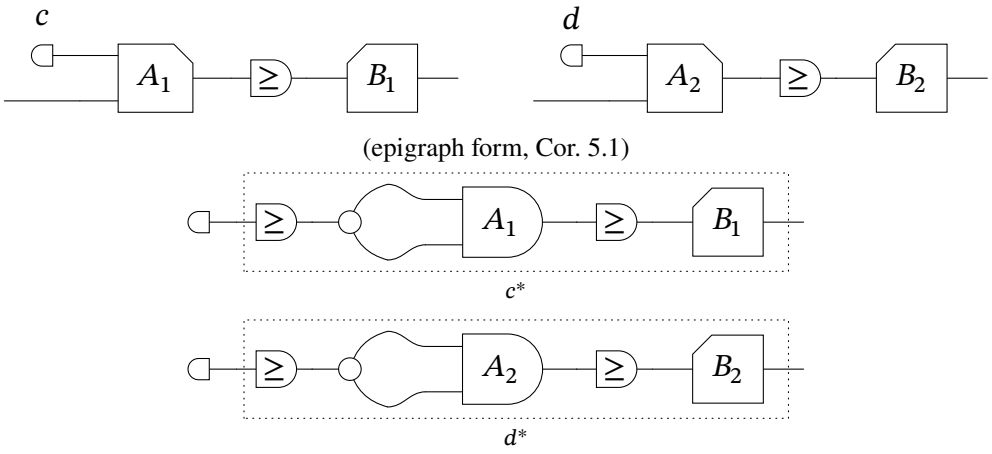
□

The final result of the thesis is the completeness theorem for **Graphical Linear Programming**. It is the most important of the three, since it is the one that makes the language actually useful for reasoning: every true equation between programs can be derived. The proof brings together all the main tools developed throughout: the epigraph form of Corollary 5.1, the shared epigraph property of Proposition 1.2, the completeness of **Graphical Polyhedral Algebra**, and the FME-based normal form.

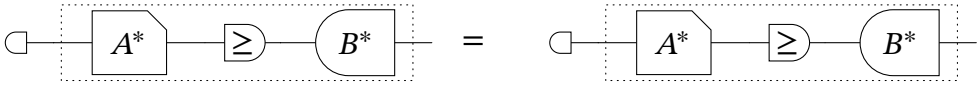
Lemma 5.4 (Completeness)**.** *If $[c] = [d]$ in* $\mathbf{LPRel}$*, then $c = d$ in* $\mathrm{Free}\mathrm{P}_{\mathbf{GLP}}$*.*

Proof. Let $v_1 = [c]$ and $v_2 = [d]$. By assumption $v_1 = v_2$, so Proposition 1.2 gives $\mathrm{epi}(v_1) = \mathrm{epi}(v_2) =: E$.

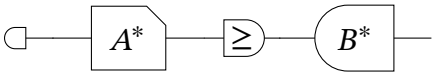
By Corollary 5.1, both c and d admit epigraph forms $c = ⊂\!\!-; c^*$ and $d = ⊂\!\!-; d^*$ with $[c^*] = [d^*] = E$ in $\mathbf{PolyRel}$. Since c^* and d^* are **Graphical Polyhedral Algebra** diagrams with the same denotation, completeness of **Graphical Polyhedral Algebra** gives $c^* = d^*$ in $\mathrm{Free}\mathrm{P}_{\mathbf{GPA}}$; in particular both reduce to the same **Graphical Polyhedral Algebra** normal form. Composing this common diagram with the leftover ⊂— collapses c and d to the same **Graphical Linear Programming** diagram:



(the dotted boxes c^*, d^* are **GPA** diagrams with $[c^*] = [d^*] = E$; by **GPA** completeness both reduce to the same normal form)



(composing the common **GPA** normal form with the leftover ⊂— yields the same **GLP** diagram)



Therefore $c = \subset\!\!-;\, c^* = \subset\!\!-;\, d^* = d$. □

As a consequence of the three lemmas above:

Theorem 5.2. **LPRel** *is presented by* $\mathrm{FreeP}_{\mathbf{GLP}}$.

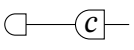
Proof. By the preceding three lemmas. □

Finally, we've built, through the addition of one generator and three axioms, an elegant and complete diagrammatic theory for parametric linear programming. This result does unlock a new algebraic and computational way to interact with LP problems. A linear program is ordinarily presented as a function to be evaluated, an $x \mapsto c^\top x$ subject to constraints, solved from the outside by an algorithm. Here it is instead a morphism, an object with an interface that composes with others of its kind, and solving it, and eliminating one of its variables are all the same operation, composition in **LPRel**.

5.3 THE COMPLETE TRANSPORTATION PROGRAM

We return to the constraint diagram of Section 4.3 and add the monetary costs $1, 2, 4$ on the roads $A \to B$, $B \to C$, and $A \to C$. The constraints stay exactly as they were; only the objective is new.

The objective is attached road by road. For a price c, the diagram



denotes the cost $(\bullet, x) \mapsto cx$: the linear generator charges its wire value t, and the reflected scalar imposes $t = cx$. We call it a *price tag*. A black node joins the price tag to the wire carrying the flow, so the flow is charged at price c without being changed. This is the same construction used for $c^\top x$ in the expressivity proof above.

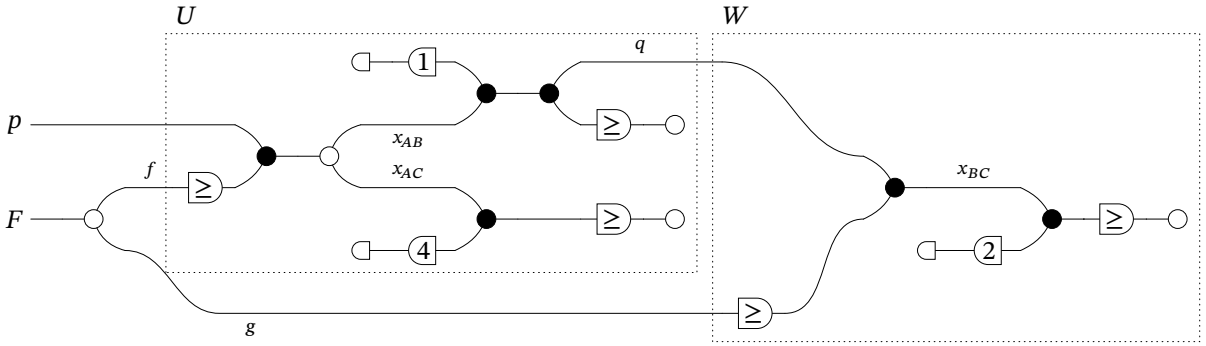


Figure 5.1 – The complete transportation program in Graphical Linear Programming. It is the constraint diagram of Figure 4.2 with one price tag on each road. The dotted boxes group the stage programs $U : 2 \to 1$ and $W : 2 \to 0$.

Compared with Figure 4.2, three price tags have been added: price 1 on x_{AB} and price 4 on x_{AC} inside the first stage, and price 2 on x_{BC} inside the second. All other nodes are polyhedral generators, which denote indicator costs: 0 when their constraint holds and $+\infty$ otherwise.

Composition in $\mathbf{LPRel}$ adds costs and minimizes over internal wires. Inside each box this gives the two stage programs of Chapter 1:

$$U(p,q,f) = \inf_{x_{AB},x_{AC}\geq 0} \{x_{AB} + 4x_{AC} \mid x_{AB} + x_{AC} = p,\ x_{AB} = q,\ p \leq f\},$$
$$W(q,g) = \inf_{x_{BC}\geq 0} \{2x_{BC} \mid x_{BC} = q,\ x_{BC} \leq g\}.$$

Composing the boxes through q, and the fuel split $F = f + g$, eliminates the internal wires f, g, and q. The whole diagram has interface $(p, F) \to *$ and denotes

$$\inf_{q,f}\big(U(p,q,f) + W(q,F-f)\big) = V(p,F),$$

which is the formula of Chapter 1. No infimum was written by hand: each one comes from an internal wire. Thus the same connections that eliminate feasibility variables in Graphical Polyhedral Algebra also minimize costs in Graphical Linear Programming.

The values can be read off the diagram. For $p = F = 1$, the split forces $f = 1$ and $g = 0$, so $q = 0$ and all traffic takes the direct road: $V(1,1) = 4$. For $p = 1, F = 2$, the choice $f = g = 1$ and $q = 1$ sends all traffic through B: $V(1,2) = 1 + 2 = 3$. For $p \geq 0$ and $F < p$, no split satisfies $p \leq f$ and $q \leq g$ with $q \geq 0$, so $V(p,F) = +\infty$.

Finally, the three price tags can be merged into one linear generator. By (linearity), two linear generators in parallel equal a single linear generator followed by a white comultiplication;

applying it twice gives a single generator whose value is split among the three prices. This is the first step towards the normal form of Theorem 5.1.

Bibliography

[Bae04] John C. Baez. *Quantum Quandaries: A Category-Theoretic Perspective*. Apr. 7, 2004. arXiv: quant-ph/0404040. URL: https://arxiv.org/abs/quant-ph/0404040.

[BDS21] Filippo Bonchi, Alessandro Di Giorgio, and Paweł Sobociński. "Diagrammatic Polyhedral Algebra". In: *41st IARCS Annual Conference on Foundations of Software Technology and Theoretical Computer Science (FSTTCS 2021)*. Vol. 213. Leibniz International Proceedings in Informatics (LIPIcs). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2021, 40:1–40:18. DOI: 10.4230/LIPIcs.FSTTCS.2021.40. arXiv: 2105.10946.

[Bon+19] Filippo Bonchi et al. "Graphical Affine Algebra". In: *Proceedings of the 34th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS 2019)*. Vancouver, BC, Canada: IEEE, 2019, pp. 1–12. DOI: 10.1109/LICS.2019.8785877.

[BSZ17] Filippo Bonchi, Paweł Sobociński, and Fabio Zanasi. "Interacting Hopf Algebras". In: *Journal of Pure and Applied Algebra* 221.1 (2017). Original complete axiomatisation of Graphical Linear Algebra, pp. 144–184. ISSN: 0022-4049. DOI: 10.1016/j.jpaa.2016.06.002. arXiv: 1403.7048.

[Coe+18] Bob Coecke et al. "Generalized Relations in Linguistics & Cognition". In: *Theoretical Computer Science* (2018). Earlier conference version in WoLLIC 2017, LNCS vol. 10388, DOI: 10.1007/978-3-662-55386-2_18. ISSN: 0304-3975. DOI: 10.1016/j.tcs.2018.03.008. URL: https://www.sciencedirect.com/science/article/pii/S0304397518301476.

[FS19a] Brendan Fong and David I. Spivak. *An Invitation to Applied Category Theory: Seven Sketches in Compositionality*. Cambridge, UK: Cambridge University Press, 2019. ISBN: 978-1-108-71182-1. DOI: 10.1017/9781108668804. arXiv: 1803.05316.

[FS19b] Brendan Fong and David I. Spivak. "Hypergraph Categories". In: *Journal of Pure and Applied Algebra* 223.11 (2019), pp. 4746–4777. ISSN: 0022-4049. DOI: 10.1016/j.jpaa.2019.02.014. arXiv: 1806.08304.

[JS91] André Joyal and Ross Street. "The Geometry of Tensor Calculus, I". In: *Advances in Mathematics* 88.1 (1991), pp. 55–112. ISSN: 0001-8708. DOI: 10.1016/0001-8708(91)90003-P. URL: https://www.sciencedirect.com/science/article/pii/000187089190003P.

[Law73] F. William Lawvere. "Metric Spaces, Generalized Logic, and Closed Categories". In: *Rendiconti del Seminario Matematico e Fisico di Milano* 43.1 (1973). Reprinted in: *Reprints in Theory and Applications of Categories*, No. 1 (2002), pp. 1–37, pp. 135–166. DOI: 10.1007/BF02924844.

[PRS22] João Paixão, Lucas Rufino, and Paweł Sobociński. “High-level axioms for graphical linear algebra”. In: *Science of Computer Programming* 218 (2022), p. 102791. ISSN: 0167-6423. DOI: 10.1016/j.scico.2022.102791. URL: https://www.sciencedirect.com/science/article/pii/S0167642322000247.

[PZ23] Robin Piedeleu and Fabio Zanasi. *An Introduction to String Diagrams for Computer Scientists*. 2023. arXiv: 2305.08768 [cs.LO]. URL: https://arxiv.org/abs/2305.08768.

[Roc70] R. Tyrrell Rockafellar. *Convex Analysis*. Vol. 28. Princeton Mathematical Series. Princeton, New Jersey: Princeton University Press, 1970. ISBN: 978-0-691-01586-6. DOI: 10.1515/9781400873173.

[Sch86] Alexander Schrijver. *Theory of Linear and Integer Programming*. Wiley-Interscience Series in Discrete Mathematics and Optimization. John Wiley & Sons, 1986.

[Sel10] Peter Selinger. “A Survey of Graphical Languages for Monoidal Categories”. In: *New Structures for Physics*. Ed. by Bob Coecke. Vol. 813. Lecture Notes in Physics. Springer, Berlin, Heidelberg, 2010, pp. 289–355. DOI: 10.1007/978-3-642-12821-9_4. arXiv: 0908.3347.

[SS24] Dario Stein and Richard Samuelson. “Towards a Compositional Framework for Convex Analysis (with Applications to Probability Theory)”. In: *Foundations of Software Science and Computation Structures (FoSSaCS 2024)*. Vol. 14574. Lecture Notes in Computer Science. Springer, Cham, 2024, ? DOI: 10.1007/978-3-031-57228-9_9. arXiv: 2312.02291.

[Ste+24] Dario Stein et al. “Graphical Quadratic Algebra”. In: *Proceedings of the 39th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS 2024)*. Springer, Cham, 2024. DOI: 10.1007/978-3-032-11176-0_18. arXiv: 2403.02284.

[Win93] Glynn Winskel. *The Formal Semantics of Programming Languages: An Introduction*. Foundations of Computing. Cambridge, Massachusetts: MIT Press, 1993. ISBN: 978-0-262-23169-5. URL: https://mitpress.mit.edu/9780262731034/.